

The Hebrew University of Jerusalem
The Rachel and Selim Benin School of Computer Science and Engineering

Provers Know When They Are Failing: Internal-State Probes for Kernel-Verified Theorem Proving

מוכיחים יודעים מתי הם נכשלים: גלאי מצב פנימי להוכחת משפטים המאומתת בגרעין

Amit Aminov

Thesis submitted in partial fulfillment of the requirements
for the Master of Sciences degree
in Computer Science

Under the supervision of **Prof. Michael Werman**

September 2026



האוניברסיטה העברית בירושלים
בית הספר להנדסה ולמדעי המחשב על שם רחל וסלים בנין
החוג למדעי המחשב

Provers Know When They Are Failing: Internal-State Probes for Kernel-Verified Theorem Proving

מוכיחים יודעים מתי הם נכשלים: גלאי מצב פנימי להוכחת משפטים המאומתת בגרעין

מוגש על ידי
עמית אמינוב

עבודת גמר לתואר מוסמך במדעי המחשב

עבודה זו הונחתה על ידי
פרופ' מיכאל וורמן

תשרי ה'תשפ"ז

תקציר

מודלי שפה גדולים (LLMs) המייצרים הוכחות עבור מערכת ההוכחות Lean מוערכים בדרך כלל על ידי דגימה של הוכחות מועמדות רבות עבור כל משפט, ושמירת כל הוכחה שגרעין ההוכחה (kernel) מקבל. משפט נחשב פתור אם לפחות ניסיון אחד מתוך K הניסיונות מאומת. לשיטה זו שתי חולשות מרכזיות. ראשית, חלק ניכר מתקציב הדגימה מבוזבז על ניסיונות שאינם סוגרים את המטרה: המודל חוזר על אותה טקטיקה עד שהוא מגיע לתקרת האסימונים. ב-MathOlympiadBench בין 2% ל-12% מן הניסיונות רצים עד התקרה באופן זה, ובניסיון אחד של מודל ה-8B מופיעה אותה למה בתוך קריאת simp יחידה 208 פעמים מבלי שההוכחה מסתיימת. שנית, אופן מדידת ההצלחה עצמו אינו אמין: הלולאה המתמשכת מסוג read–eval–print (REPL) הנפוצה לניקוד הוכחות שופטת ניסיון לפי צורת התשובה של השרת ואינה בודקת כלל את האקסיומות שעליהן ההוכחה נשענת, כך ש"הוכחה" עצמאית הנשענת על אקסיומה שמחוץ לרשימה המהימנה מתקבלת כחיובי כוזב.

עבודה זו שואלת האם המצב הפנימי של המוכיח, המחושב ממילא במהלך הייצור הרגיל, מסמן כשל מוקדם דיו כדי להיות שימושי, ומקפידה למדוד כל תוצאה מול הגרעין של Lean. אנו מאמנים שני גששים קטנים (מסווגי כשל): ראש MLP וראש LSTM, כל אחד בפחות מ-3M פרמטרים, הקוראים את המצב הנסתר של מוכיח קפוא בין שורות ההוכחה ומנבאים האם הניסיון הנוכחי ייכשל. התוויות (labels) מגיעות מפרוטוקול אימות איתן המהדר כל הוכחה מחדש בתהליך Lean נקי ומקבל אותה רק אם היא עוברת בדיקת טיפוסים ללא sorry ואינה נשענת על דבר מעבר לאקסיומות הגרעין

המהימנות. מכיוון שכל משפט אימון הוא למה מתוך הספרייה שהסביבה הבודקת כבר מכילה, אנו גם מסננים את קבוצת האימון למשפטים שניסוחם המשוחזר מתקמפל, ומתייגים כל ניסיון תחת פרוטוקול שמופעל בזמן האימות בלבד ומסיר את הציטוט הישיר של הלמה ואת רישומי האוטומציה שלה. הסתברות הכשל של הגשש מוזנת חזרה אל שלב הייצור (generation) כדי להכוון מוכיח Goedel-Prover-V2 קפוא (משקוליו אינם מתעדכנים כלל), באמצעות הכפלת הלוגיטים של האסימון הבא ב-$e^{-\alpha p_{\text{fail}}}$ (העלאת טמפרטורה בכל תחילת שורה) או הוספת עונש תדירות המשוקלל בהסתברות הכשל. תחרות על קבוצת האימות, מעל רשת פרמטרים משותפת לשני הראשים, בוחרת את ההגדרה, ואותה הגדרה מנצחת בשני מודלי הבסיס.

אנו מעריכים על MathOlympiadBench (360 משפטים אולימפיים), תמיד ב-pass@32 ותמיד תחת הפרוטוקול האיתן. המדידה קודמת לכל: תחת הפרוטוקול שלנו ה-pass@32 המדווח של Goedel-Prover-V2 על MathOlympiadBench עבור 32B משוחזר במדויק ($60 = 60$), ואנו מתעדים הוכחה קונקרטית של False שמנקד REPL מתמשך מקבל והגרעין דוחה. האות אמיתי, והוא מוקדם: הגששים מגיעים ל-ROC AUC של 0.89–0.92 באימות צולב בטוח-מדליפה, ו-0.81–0.86 בהעברה אל מחוץ לתחום, ובחלונות של 250 עד 4,000 האסימונים הראשונים הם מגיעים ל-0.64–0.79 בעוד שהמאפיין המתבקש, כמה אסימונים נוצרו עד כה, עדיין נמצא ברמת ניחוש אקראי. לעומת זאת, ההכוונה על בסיס אות זה אינה עובדת: היא פותרת 6 משפטים פחות מן הבסיס ב-8B ומשפט אחד פחות ב-32B, והיא עולה כשישית מכלל הניסיונות המאומתים בשני מודלי הבסיס. בתוך התחום, על משפטי mathlib שעליהם נבחרה ההגדרה, אותה הגדרה דווקא שיפרה את התוצאה ב-1+ וב-6+ משפטים; היפוך זה הוא המדד הברור ביותר שיש לנו לפער התחומים, והסיבה לכך שהצעד הבא שהיינו נוקטים הוא לאמן ולכייל את הגשש בתחום שבו הוא משמש בפועל. אנו מראים מדוע, ולא רק מדווחים על התוצאה: ההתערבות אינה חלשה מדי, שכן היא אכן מקצרת את הייצור ומפחיתה חזרתיות במידה ניכרת, אך היא חסרת כיוון: p_{fail} קובע את עוצמתה של הפרעה קבועה שאינה תלויה בתוכן, ולכן השינויים שהיא מחוללת אינם דחיפה אל עבר הצורה שמצליחה, ובסך הכול הם עולים ביותר ניסיונות מאלה שהם מצילים;

המדיניות שאותה אנו מנסים לשנות היא גם תוצר של למידת חיזוק בקנה מידה גדול.

תרומות העבודה הן אפוא שלוש. מדידה, אימות ותיוג: ביקורת ושחזור איתנים ובלתי תלויים של מוכיח פתוח מוביל, לצד תיאור של אופן תיוג נתוני האימון הנגזרים מספרייה ואמירה מפורשת על מה שתיוג כזה אינו יכול להסיר. אות כשל מוקדם מבוסס מצב פנימי: שני ראשי גשש המאומנים על מצביו הפנימיים של מוכיח קפוא מתוך פסיקות הגרעין, לצד ראיות לכך שמצבים אלה מנבאים כשל מוקדם, מחוץ לתחום, ולא רק כמשתנה עקיף לאורך הניסיון. תוצאה שלילית: הזנת הסתברות הכשל הזו חזרה אל הייצור, כאשר ההגדרה נבחרת בתחרות מעל רשת משותפת לשני הראשים, אינה משפרת את המוכיח, תחומה במדויק, יחד עם המנגנון שמאחוריה.

Abstract

Large language models (LLMs) that write proofs for the Lean proof assistant are usually evaluated by *sampling* many candidate proofs per theorem and keeping any that the Lean kernel accepts. This procedure exposes two weaknesses. First, much of the sampling budget is spent on attempts that never close the goal: the model repeats a single tactic long after it has stopped making progress. On MathOlympiadBench, 2–12% of attempts run all the way to the token cap, and one attempt of the smaller model we examine names the same rewrite lemma 208 times inside a single `simp` call without ever finishing. The cap understates the waste: on that model, 24.4% of failed attempts run past the longest attempt that ever verified, consuming 49.2% of all generated tokens. Second, the way success is measured is itself unreliable: the persistent read–eval–print loop (REPL) commonly used to score proofs judges an attempt from the shape of the server's reply and never inspects the axioms it depends on, so a self-contained "proof" that rests on an off-whitelist axiom is accepted as a false positive.

This thesis asks whether a prover's own internal state, computed for free during ordinary generation, signals failure early enough to be useful, and it insists on measuring every result against the Lean kernel. We train two small probes, a multi-layer perceptron (MLP) head and a long short-term memory

(LSTM) head, each under 3M parameters, that read the frozen prover's hidden state between proof lines and predict whether the current attempt will fail; the labels come from a *robust verification protocol* that recompiles each proof in a fresh Lean process and accepts it only when it type-checks with no `sorry` and depends on nothing beyond the trusted kernel axioms. Because every training theorem is a library lemma that the checking environment already contains, we also filter the training set to theorems whose reconstructed statement elaborates and label each attempt under a verification-time protocol that removes the target lemma's own citation and automation entries. The probe's failure probability is then fed back into generation to steer a *frozen* Goedel-Prover-V2, in its released 8B and 32B sizes (its weights are never updated), either by raising the sampling temperature to $e^{\alpha p_{\text{fail}}}$ at each line start or by adding a probe-scaled frequency penalty; a validation contest over a grid shared by both heads selects the setting, and the same one wins on both base models.

We evaluate on MathOlympiadBench (360 olympiad theorems), always at pass@32 and always under the robust protocol, and we report the three contributions in that order. The measurement comes first: under our protocol Goedel-Prover-V2's reported 32B MathOlympiadBench pass@32 reproduces exactly ($60 = 60$), and we document a concrete proof of `False` that a persistent-REPL scorer admits and the kernel rejects. The signal is real and it is early: the probes reach ROC-AUC 0.89–0.92 in leakage-safe cross-validation and 0.81–0.86 transferred out of domain, and in windows from the first 250 to the first 4,000 generated tokens they reach 0.64–0.79 where the obvious surface feature, how many tokens have been generated so far, is still at chance. Steering on that signal at each proof-line start, however, does not work: it solves 6 fewer theorems than the base on 8B and 1 fewer on

32B, and it costs about a sixth of all verifying attempts on both base models. In domain, on the held-out mathlib theorems the setting was chosen on, the same arm had *gained* +1 and +6 theorems; that reversal is the clearest measure we have of the domain gap, and the reason the first thing we would try next is to train and calibrate the probe in the domain it is used in. We show why rather than only reporting the outcome: the intervention is not too weak, since it shortens generations and cuts repetition markedly, but it is undirected, and the changes it makes are not a nudge toward the shape that succeeds, costing more attempts than they rescue; the policy being edited is also the product of large-scale reinforcement learning.

The contributions are thus threefold. *Measurement, verification and labeling:* an independent, kernel-checked audit and reproduction of a state-of-the-art open prover, together with an account of how to label library-derived training data and an explicit statement of what such labeling cannot remove. *An early internal-state failure signal:* two probe heads trained on a frozen prover's internal states from kernel verdicts, with evidence that those states predict proof failure early, out of domain, and not merely as a proxy for how long an attempt has run. *Negative steering result:* feeding that failure probability back into generation, with the setting chosen by a contest over a grid shared by both heads, does not improve the prover, bounded precisely, together with the mechanism behind it.

Acknowledgements

I am grateful to my advisor, Prof. Michael Werman, for his guidance, patience, and support throughout this research. I thank the members of the School of Computer Science and Engineering at the Hebrew University of Jerusalem for the environment that made this work possible, and the maintainers of the Lean and mathlib communities, whose open tools underlie every experiment in this thesis. Finally, I thank my family and friends for their encouragement.

Contents

i **תקציר**

Abstract **iv**

Acknowledgements **vii**

1 Introduction **1**

1.1 Formal theorem proving with language models 1

1.2 Two problems with sample-and-check 2

1.3 This thesis 4

2 Motivation **6**

2.1 The cost of doomed attempts 6

2.2 The unreliability of persistent-REPL scoring 7

2.3 Bringing the two together 8

3 Previous Work **9**

3.1 Inference-time control of frozen language models 9

3.2 Online prediction of the outcome from hidden states 10

3.3 Learned guidance for proof search 11

3.4 Failure-aware and retry-based proving 11

3.5 The compute-heavy and agentic frontier 12

3.6 The reliability of formal-proof evaluation 12

3.7 Positioning 13

4 Research Questions **14**

5 Methods **17**

5.1 Robust verification protocol 17

5.2 Overview 19

5.3 Probe inputs and heads 19

5.4 Where we read, and why there 20

5.5 Labels and training 20

5.6 Steering interventions 22

5.7 Selection (validation contest) 22

5.8 Base architecture and hidden-state extraction 24

5.9 Training data 24

5.10 Prompt 25

5.11 Sampling and normalization 26

6 Results **27**

6.1 Setup 27

6.2 Reproducing the Goedel-Prover-V2 baseline 28
6.3 Probe generalization 29
6.4 The signal 30
6.5 Main results 32
6.6 Validation and test disagree 33
6.7 Which theorems change 34
6.8 Why this intervention does not help 34

7 Discussion and Conclusion 36

7.1 Summary of findings 36
7.2 Limitations 37
7.3 Future directions 40
7.4 Concluding remarks 41

A Examples of Repetition 50

B Filtering and Target-Blinded Labeling 53

B.1 What the probe learns from 54
B.2 Why the raw labels were unusable 54
B.3 Filtering: keep statements that elaborate 54
B.4 Target-blinded labeling 56
B.5 What counts as a label 57
B.6 Why each guard is needed 60
B.7 Why the citation filter matches positions, not mentions 60

B.8 The attribute-error guard, and what it does not catch 64

B.9 What still leaks 66

B.10 Prompt and elaboration environment: four options 67

C Positioning Against the Frontier **75**

C.1 The measurement contract, independently adopted 76

D Supplementary Background and Provenance **79**

D.1 How the base prover was trained 79

D.2 Outline of the thesis 82

D.3 The approach in brief 82

D.4 Extended motivating discussion 83

D.5 Extended related-work detail 84

E Full Methods and Implementation **86**

E.1 The Lean file a persistent REPL accepts 87

E.2 Pipeline and probe-head figures 88

E.3 Trainable parameters 91

E.4 Tap-point statistics 91

E.5 The epoch-selection score, and why it is not a result 92

E.6 The training loop 92

E.7 Why this α grid 93

E.8 Steering grid and validation contest 95

E.9 Base architecture and the probe tap 96

E.10 Base-architecture internals and extraction 97

E.11 Probe training data 98

E.12 The prompt, exactly as sent 99

E.13 Normalization and run metadata 100

E.14 Reproducibility and released artifacts 101

F Supporting Empirical Analyses 102

F.1 Probe diagnostics 102

F.2 Where generalization reaches, and where it stops 104

F.3 Compressibility and the shape of the waste 105

F.4 Off-the-shelf classifiers 106

F.5 Validation and test, in detail 106

F.6 Which theorems change, case by case 107

F.7 Mechanism and divergence analyses 108

G Supplementary Discussion 113

G.1 Extended limitation details 113

G.2 Extended future-direction detail 116

List of Figures

6.1 The signal is available early; length is not 31

E.1 Probe-guided generation pipeline 89

E.2 MLP probe head 89

E.3 LSTM probe head 90

E.4 Base architecture and probe tap 97

F.1 Probes separate success from failure early, then collapse 103

F.2 Failed attempts run far longer than any that succeeds 104

F.3 Steering shortens successful proofs and makes them less repetitive 111

F.4 Steering moves failing generations away from success 112

List of Tables

6.1 MathOlympiadBench reproduction 29

6.2 Probe failure-prediction ROC-AUC 30

6.3 Robustly verified pass@32 33

B.1 Theorem filtering 55

B.2 Two naive alternatives to the citation filter. The first column is measured on the 3,398 positive labels, the second on the 122 citations the deployed filter removed. 61

B.3 The attribute-error guard on the cases that arose, and on the case it is for. 65

B.4 Four pairings of prompt and elaboration environment 74

C.1 Representative Lean provers 78

E.1 Trainable parameters 91

E.2 Effective temperature of the temperature intervention 95

E.3 Validation contest results 96

E.4 Probe training data 99

Chapter 1

Introduction

1.1 Formal theorem proving with language models

A *formal* proof is one written in a language precise enough that a computer can check it with no human judgment. In this thesis the language is Lean 4 and its mathematical library mathlib [11]: a statement is a typed term, a proof is a program that constructs a term of that type, and a small trusted *kernel* decides whether the term is well typed. If the kernel accepts, the theorem is proved, full stop; there is no partial credit and no appeal to plausibility. This binary, machine-checkable notion of correctness is what makes formal proving attractive as a testbed for machine reasoning: unlike natural-language mathematics or numeric benchmarks, a formal benchmark cannot be gamed by a fluent but wrong answer.

Modern systems attack this task by fine-tuning a large language model (LLM) to emit Lean proof scripts and then *sampling* many candidate proofs per theo-

rem, keeping any one the kernel accepts. A theorem is considered *solved* once any of its sampled attempts passes, and the standard metric is pass@k: the fraction of theorems solved within k samples. This whole-proof, sample-and-check paradigm underlies most current provers, including BFS-Prover [35], Goedel-Prover-V2 [23], DeepSeek-Prover-V2 [29], and Seed-Prover [8]. The base model in this thesis is the openly released Goedel-Prover-V2, in its 8B and 32B sizes; we use it *frozen* throughout, never updating its weights. That choice matters: the method needs a prover we can open, and one whose size leaves the full protocol affordable without giving up capability. Section 5.8 sets out the argument and the evidence for it.

1.2 Two problems with sample-and-check

The sample-and-check paradigm is simple and effective, but it hides two problems that this thesis takes as its starting point.

Wasted search budget. Sampling is expensive, and a large share of the budget is spent on attempts that were doomed early. A common failure mode is *repetition*: the model emits the same tactic, or the same short block of tactics, over and over without ever making progress toward closing the goal, sometimes until it hits the token cap. On MathOlympiadBench, 2–12% of attempts show this pattern. One 8B base sample names a single lemma 208 times inside one `simp` call (Appendix A). These attempts consume the same compute as a genuine attempt but can never succeed. The cap, though, is the visible part of the waste rather than the large part. Only 12.1% of 8B attempts reach it at all, while 24.4% run past the longest attempt that has ever verified on this benchmark, and those consume 49.2% of every token the

model generates (Figure F.2; on 32B the same figures are 1.8%, 5.4% and 10.9%). Half the 8B sampling budget is spent beyond the point where success has never once been observed. A natural question is whether such attempts can be recognized *early* and abandoned, freeing budget for fresh samples. Simple surface heuristics do not suffice, because the prover's whole method is to split a hard goal into many similar sub-goals, so repetition of a tactic is not *per se* a sign of failure. We therefore ask a deeper question: does the model's own internal state, as it generates, already contain the information needed to tell a doomed attempt from a promising one?

Unreliable measurement. The second problem is more subtle and, in a sense, more dangerous: the way proof success is usually *measured* is not sound. For speed, pipelines often score a candidate proof through a long-running Lean server, a persistent read–eval–print loop (REPL), and judge the attempt from the shape of the server's reply. Such a scorer never runs `#print axioms`, the Lean command that reports which axioms a finished proof depends on, so it never inspects what the proof actually used. As a result, a self-contained proof that leans on an axiom outside the trusted set, for instance `native_decide` together with an `@[implemented_by]` override, or a hand-written `axiom`, can prove even a false statement and still be scored as complete. This is not a hypothetical: we exhibit a proof of `False` that a persistent-REPL scorer accepts and the kernel's axiom check rejects (Chapter 5). If the yardstick admits false positives, then reported gains, including our own, are meaningless unless re-measured against the kernel.

1.3 This thesis

This thesis addresses both problems at once. To the first, we respond with a *learned internal-state signal*: a small probe, trained on the frozen prover's hidden states, that predicts whether the current attempt will fail. To the second, we respond with a *measurement contract*: every label used to train the probe and every number reported in the thesis is produced by a robust verification protocol that recompiles the proof in a fresh Lean process and accepts it only when the kernel type-checks it with no `sorry` and with an axiom set inside the trusted whitelist. The two responses reinforce each other, since the probe is only as trustworthy as the labels it learns from.

Contributions.

1. **Measurement: verification and labeling.** We accept a proof only when a fresh Lean process compiles it and the axioms it rests on stay inside the trusted kernel set. Under that protocol Goedel-Prover-V2's reported 32B MathOlympiadBench pass@32 reproduces *exactly* ($60 = 60$), and we document a concrete proof of `False` that a persistent-REPL scorer accepts but the kernel rejects. The training side needs the same discipline: every training theorem is a mathlib lemma, so the environment that checks a proof already contains the answer and a correct proof is recorded as a failure. We therefore filter to theorems whose reconstructed statement elaborates, then label each attempt under a verification-time scheme that renames the head, switches off the target lemma's own automation entries, and drops proofs that cite it, leaving labels that are leakage-reduced rather than leakage-free (Chapter 5, Appendix B).

2. **An early internal-state failure signal.** We train MLP and LSTM failure probes on a frozen prover's internal states from kernel-verified labels, with no reinforcement learning, tools, or agentic loop (Chapter 5). The signal they read is real and it is early: on held-out folds split by theorem they reach ROC-AUC 0.89–0.92, transferred out of domain from mathlib to olympiad problems they still reach 0.81–0.86, and, more tellingly, restricted to windows from the first 250 to the first 4,000 generated tokens they reach 0.64–0.79 while the obvious surface feature, how many tokens have been generated so far, is still at chance (Chapter 6).

3. **Negative steering result.** We feed the predicted p_{fail} back into generation, smoothing the next-token distribution at each line start by one of two rules, and pick the setting by a contest over a grid shared by both heads (Chapter 5). Under the robust protocol, steering solves 35 of 360 against the base's 41 on 8B, and 59 against 60 on 32B. Neither difference is significant at k=32, the budget the comparison was designed around, so we report a negative result rather than a gain. The consistent effect is at the attempt level: steering costs about a sixth of all verifying attempts on both base models. We also give the mechanism behind it, rather than only reporting that it did not work (Chapter 6).

Chapter 2

Motivation

This chapter develops the two problems introduced in Chapter 1: the compute wasted on attempts that cannot succeed, and the unreliability of the measurement used to decide success. Together they motivate the two halves of the thesis, a learned failure signal and a robust verification contract.

2.1 The cost of doomed attempts

Because a single sampled proof rarely succeeds, provers spend most of their budget generating attempts that fail. Many of these failures are not near-misses that ran out of tokens on the last useful step; they are attempts that stopped making progress early and then filled the remaining budget with repetition. The clearest form is a tactic, or a short block of tactics, emitted again and again with the proof state never advancing, sometimes until the token cap is reached and often stopping well before it. Appendix A shows two real examples the base loses to repetition: an 8B attempt piles the lemma `Finset.sum_range_succ` into a single `simp` call 208 times, and a 32B

attempt cascades a near-identical `have/nlinarith` block 38 times. Neither base attempt closes its goal, and only the second is recovered by the steered model. Neither reaches the token cap either: they stop at 20,422 and 17,827 generated tokens, about half the 40,960 budget.

Two facts make this worth addressing. First, it is common: 2–12% of MathOlympiadBench attempts reach the token cap in this way. Second, it is detectable in principle long before the cap, because a human reading the transcript can see within a few lines that the attempt has stalled. If the prover could make the same judgment from its own state, the wasted budget could be reallocated to fresh samples, raising pass@k at fixed cost. This is the practical promise behind the failure probe, and although in this thesis we use the probe to *steer* generation rather than to stop it, the same signal is exactly what an early-stopping budget controller would need (Chapter 7).

2.2 The unreliability of persistent-REPL scoring

Even if we improve which proofs are generated, the results are meaningless unless success is measured soundly. Here the field has a widespread problem. To score proofs quickly, pipelines commonly keep a Lean server alive as a persistent read–eval–print loop (REPL) and feed candidate proofs to it one after another, judging each from the shape of the server's response. Such a scorer decides "complete" from the presence or absence of error messages; it does not run `#print axioms` and does not inspect the axiom set the proof depends on.

This gap matters because Lean's soundness guarantee is about its *kernel* and

its *trusted axioms*. A proof term is only as trustworthy as the axioms it uses. Certain tactics introduce axioms outside the trusted set: `native_decide`, for instance, trusts a compiled decision procedure and records the axiom `Lean.ofReduceBool`. Ordinarily this is benign, but combined with an `@[implemented_by]` override, which lets the compiled value of a definition disagree with the value the kernel sees, it can be turned into a proof of a false statement. A persistent-REPL scorer, seeing no error, marks such a proof complete; the kernel's axiom check rejects it. Chapter 5 gives a concrete, self-contained proof of `False` that exhibits exactly this behavior.

2.3 Bringing the two together

The two problems are complementary. The first is about *generation*: how to spend a fixed sampling budget on attempts more likely to succeed. The second is about *evaluation*: how to know, reliably, whether an attempt succeeded. A learned signal that improved generation but was scored by an unsound yardstick would prove nothing; conversely, a sound yardstick alone does not make a prover better. This thesis pairs a small internal-state probe that steers generation with a kernel-checked contract that measures it, so that any improvement we report is both real and reliably observed.

Chapter 3

Previous Work

This thesis sits at the intersection of two literatures: inference-time control of language models, and machine learning for formal theorem proving. It also draws on a third, smaller literature on the reliability of formal-proof evaluation. We review each in turn and then position our contribution.

3.1 Inference-time control of frozen language models

A body of work steers a fixed language model at inference by reading or perturbing its internal computation rather than retraining it. PPLM backpropagates gradients from an attribute classifier into the model's activations to bias generation [9]. FUDGE, the closest precedent to our method, reweights the next-token logits using a discriminator trained on partial sequences, controlling generation without touching the base weights [37]. A parallel line reads hidden states with linear *probes* to test what information they en-

code [1], and uses probe-identified directions to steer behavior, for example toward truthfulness, via inference-time intervention and representation engineering [5, 19, 39]. Our probe is in this family: it is a small classifier on a frozen model's hidden states whose output edits the logits. Closest in domain, Kirtania and Iyer [14] steer a frozen Lean prover along unsupervised activation directions; they use no trained classifier and no kernel supervision, where our probe is fitted on kernel verdicts and its labels are decided by the proof assistant. The essential difference is the target. Those methods steer toward a *soft, unverifiable* attribute (sentiment, "truthfulness"); ours steers toward a *kernel-verifiable* outcome, and its labels are decided by a proof assistant rather than by human ratings.

3.2 Online prediction of the outcome from hidden states

The idea that a model's own hidden states say something about how its generation will *end*, early enough to act on, has clear precedent outside formal proving, and our probe belongs to that family.

The last of these is the nearest precedent for our *method*, not merely our motivation: the boundary readout, the terminal label broadcast to every boundary, and the class-weighted loss are the same choices we make (§5.3). Three things differ, and they are what this thesis is about. Their supervision is whether a natural-language answer was correct, ours is a Lean kernel verdict on a whole proof, which is decidable rather than judged. Their boundaries are reasoning delimiters, ours are proof lines in generated Lean. And their action *ends* a generation, where ours edits the next-token distribution of the

generation that continues. We take none of this as diminishing the question; it does mean the shared recipe is a starting point we inherited rather than a contribution we are claiming.

3.3 Learned guidance for proof search

A second line attaches learned guidance to the states or transitions of a proof search. ReProver retrieves premises and guides tactic selection in Lean [36]; LeanProgress predicts progress toward closing a goal [12]; 3D-Prover scores candidate tactics [16]; HTPS runs a hypertree proof search guided by a learned policy and value [17]; and rlCoP applies reinforcement learning to connection-tableau search [13]. These methods operate on the *proof-state graph*, scoring tactics or states inside a search procedure. Our probe instead reads the *generator's* hidden state during whole-proof generation and never inspects the Lean proof state, which keeps it cheap (no extra forward pass, no environment interaction) at the cost of a coarser signal.

3.4 Failure-aware and retry-based proving

Several systems learn from failed attempts, but they act *between* attempts. Learn-from-Failure trains on unsuccessful trajectories [3]; APRIL refines proofs across rounds [32]; and APOLLO repairs proofs using compiler feedback [28]. Our probe acts *within* a single attempt, editing the logits as the proof is written, and is orthogonal to between-attempt refinement: a probe-steered generator could feed any of these outer loops.

3.5 The compute-heavy and agentic frontier

The strongest current provers reach their results with much larger parameter, search and interaction budgets than we use: reinforcement-trained provers that scale test-time sampling and refinement, and tool-using Lean agents that iterate against the compiler many times (Appendix D.5). We deliberately occupy the opposite, low-budget corner: a frozen open base, a small sampling budget, and one sub-3M-parameter probe, with no reinforcement learning, tools, or agentic loop. We make no state-of-the-art claim; the point is to isolate the effect of a cheap internal-state signal and to establish the measurement discipline such a claim would require.

3.6 The reliability of formal-proof evaluation

A final, methodological literature documents that formal-proof *evaluation* is itself error-prone. Faults in formal benchmarking [2], SorryDB's study of incomplete proofs [18], LeanTree's discovery of REPL-accepted but incorrect MiniF2F proofs [15], and catalogues of Lean evaluation pitfalls [31] all point to the same failure: scoring a proof by the response of a long-running server rather than by the kernel's axioms admits false positives. Our robust verification protocol (Chapter 5) is a direct response, and our reproduction audit (Chapter 6) quantifies the gap for one released prover.

3.7 Positioning

In one sentence: we bring FUDGE-style logit steering [37] to a frozen formal-theorem prover, drive it with a failure probe trained on *kernel-verified* labels, and measure every result under a robust verification contract that the reliability literature shows to be necessary. To our knowledge this specific combination, internal-state steering of a frozen prover under a kernel-checked contract, has not been studied before. The claim is deliberately narrow. Predicting an outcome from hidden states mid-generation, and acting on that prediction, is established work [10, 21, 38]; what we have not found elsewhere is a formal prover, kernel verdicts as the labels, and an action that edits the generation instead of ending it.

Chapter 4

Research Questions

The two problems of Chapter 2 lead to four research questions, grouped the way the rest of the thesis groups them. The first concerns the measurement contract that all the others depend on, and Chapter 6 answers it first. The second and third concern the central hypothesis, that a frozen prover's internal state is a useful failure signal and that the signal reaches beyond the domain it was fitted in; they are answered together, in §6.3 and §6.4. The fourth asks whether that signal can steer the prover, and it is answered last.

Q1 (Measurement). *How reliable are current persistent-REPL evaluations, and how do robust, kernel-checked scores compare?* Because the other three questions all rest on trustworthy labels and trustworthy scores, we establish first how much the usual evaluation can be trusted. We hypothesize that a persistent-REPL scorer admits demonstrable false positives, and that a released prover's reported scores must be recomputed under a robust protocol before they can be trusted. Success criterion: a concrete, self-contained proof that the REPL accepts and the kernel rejects, plus a like-for-like robust repro-

duction of Goedel-Prover-V2's reported pass@32 on MathOlympiadBench.

Q2 (Signal). *Does a frozen prover's mid-proof internal state predict whether the current attempt will fail?* We hypothesize that the hidden state the model computes at proof-line starts already separates attempts that will be kernel-accepted from those that will not, well enough that a small classifier can recover it. Success criterion: a probe trained on kernel-verified labels achieves ROC-AUC clearly above chance (0.5) on held-out theorems.

Q3 (Generalization). *Does the learned failure signal transfer out of domain?* The probes are trained and selected on LeanDojo/mathlib theorems, but the test benchmark is olympiad-level (MathOlympiadBench), which differs sharply from mathlib in style and difficulty. We hypothesize that the failure signal is not a mathlib artifact and continues to discriminate failure on this unseen domain. Success criterion: out-of-domain per-line ROC-AUC remains above chance on the test benchmark.

Q4 (Steering). *Can the predicted failure probability, fed back into generation, steer a frozen prover toward more robustly verified proofs at a fixed sampling budget?* We hypothesize that discouraging the token patterns that precede failure, by raising the effective temperature or penalizing repeated tokens in proportion to the predicted failure probability, shifts probability mass toward attempts that verify. Success criterion: at equal pass@32 budget, a probe-steered model solves more theorems than the unsteered base on the test benchmark, under the robust protocol, with the base weights unchanged.

Scope. We restrict attention to a single frozen open base prover (Goedel-Prover-V2, 8B and 32B), whole-proof generation at pass@32, and two lightweight probe architectures with two generation interventions. We do not fine-tune the base, add tools or retrieval, or run an agentic loop; these are deliberately out of scope so that the effect of the internal-state signal can be isolated. Chapters 5 and 6 answer them with measurement first and steering last; Q2 and Q3 are taken together in between.

Chapter 5

Methods

This chapter defines the measurement contract and the method. We begin with the robust verification protocol (§5.1), because it produces the labels the probes learn from and the scores we report. We then give an overview of the pipeline (§5.2), the probe inputs and architectures (§5.3), the training objective (§5.5), the steering interventions (§5.6), the selection contest (§5.7), the base architecture and how we extract hidden states (§5.8), how that base was itself trained (§D.1), the training data (§5.9), the released prompt we decode from (§5.10), the sampling and normalization details (§5.11); what we release is listed in Appendix E.14.

5.1 Robust verification protocol

Proof-generation pipelines commonly score a candidate proof by the response of a long-running Lean server (a persistent read–eval–print loop, or REPL) rather than by the kernel's axioms, which is unreliable. The persistent-REPL scorer released with Goedel-Prover-V2 judges a proof from the shape of the

REPL's response, which never runs `#print axioms` or inspects the axiom set, so a self-contained proof resting on an off-whitelist axiom (`native_decide`, or a hand-written `axiom`) passes as a false positive. This gap between what the REPL accepts and what the kernel proves is not specific to one system; it recurs across the field [2, 15, 18].

We instead check every attempt against the kernel in a fresh process, in four steps.

1. We *reject* placeholder or malformed strings (empty, `None`, error markers), and any source that itself contains `sorry`, before compiling anything; the compiler's own `declaration uses 'sorry'` warning is caught separately at step 4.
2. We append `#print axioms <thm>` to the Lean file.
3. We compile it *in a fresh Lean process* with `lake env lean` under mathlib4 (Lean 4.9.0-rc1) [11].
4. We *reject* on a nonzero exit code, any `error:`, a `declaration uses 'sorry'` warning, or an axiom set not contained in {`propext`, `Classical.choice`, `Quot.sound`}.

A proof attempt is considered successful only if it passes all four conditions, which confirms that the proof term type-checks under the trusted kernel axioms in the audited environment. Every label and every reported number in this thesis uses this protocol.

A false positive the REPL accepts. A self-contained Lean file can "prove" `False` by exploiting a mismatch between a definition's kernel value and its compiled value; Appendix E.1 gives that file and its explanation. A persistent-REPL scorer sees no error and marks the proof complete.

Our protocol runs `#print axioms bad`, finds the off-whitelist axiom `Lean.ofReduceBool`, and rejects it. This single example is the concrete justification for step (4): without an axiom check, a scorer can accept a proof of a falsehood.

5.2 Overview

The pipeline has three parts (Figure E.1). A small **failure classifier** reads the prover's internal states. A generation-time **steering** rule turns its score into a change in the logits. A validation **contest** then selects the best probe and intervention combination. We train only the classifier; the base prover stays frozen and receives zero parameter updates.

5.3 Probe inputs and heads

Let $y_{1:T}$ be a generated attempt with indentation-token positions $b_1, b_2, \ldots$. At each line start b_t we save one vector: the frozen model's internal state after its *final* normalization layer (`model.model.norm`). This vector has dimension H (4096 for 8B, 5120 for 32B), and we apply feature normalization using the mean and standard deviation computed on the training data. The *MLP* head is stateless: its input is the normalized pair $[h_{t-1}; h_t]$, and it applies LN $\to$ $2H \to 256 \to$ ReLU $\to$ drop $\to 1 \to \sigma$, where LN is layer normalization and ReLU the rectified-linear activation (Figure E.2).

The *LSTM* head is stateful: it reads one line start at a time with normalized input h_t and applies LN $\to$ $H \to 128 \to$ ReLU $\to$ LSTM(128) $\to$ LN $\to$ $64 \to 1 \to \sigma$, updating its hidden and cell state across line starts within an

attempt and resetting at the start of each new attempt (Figure E.3). The generator sees only p_{fail}.

5.4 Where we read, and why there

A proof line in the Lean block begins with its indentation, so the indentation token that opens the line is the last position before the line's actual content. Reading there gives the probe the model's state at the moment it has committed to starting a line but not yet to what the line says, and that is precisely the decision the steering rule of §5.6 is meant to influence: one token later the tactic has already been chosen.

5.5 Labels and training

Each saved line start takes the robust-verification outcome of the attempt it belongs to, $z_t := \mathbb{1}[\text{attempt fails}]$ (§5.1); all line starts of one attempt share the same label. What exactly counts as a positive and as a negative label, and why the negative rule requires failure even when the target lemma is fully available, is set out formally in Appendix B.5. We train on complete generations from the frozen base with a weighted binary cross-entropy (wBCE) loss, giving more weight to successes to offset the failure/success imbalance. This recipe is not new, and we do not present it as ours: Liang et al. [21] score reasoning-step boundaries the same way, propagating a trace's final correctness label to every step in it and fitting a small classifier under a class-weighted binary cross-entropy. What is different here is the supervision and the setting. Their label is whether an answer was correct; ours is a Lean kernel verdict on a whole proof, and our boundaries are proof lines rather

than reasoning delimiters. Table E.1 lists the trainable parameter counts: the base model is used as released and only the small head is trained.

5.5.1 Three ROC-AUCs, and what each one measures

This thesis reports two ROC-AUCs and computes a third that it does not report, and they are easy to conflate, so it is worth separating them once. Two of the three do not describe the head we ship.

1. **The epoch-selection score** is the one computed after every epoch to choose which epoch to keep. It is a training control, not a result, and is not comparable with either number below; Appendix E.5 explains why.
2. **The cross-validated score**, 0.89–0.92, refits a head on each of four theorem-grouped folds of the 243 training theorems and scores the fold held out (§6.3). Every theorem is tested exactly once. Because each fold trains its own head, this estimates *the procedure* in domain; it never scores the checkpoint we deploy.
3. **The out-of-domain score**, 0.81–0.86 on MathOlympiadBench, is the one that scores the deployed head itself, on 360 theorems disjoint from everything it was fitted on.

The released checkpoints carry the three under separate names, so the distinction survives outside this text. We report every ROC-AUC to two decimals throughout, because the fold-to-fold spread is 0.02 to 0.04 and a third decimal would imply a precision the estimate does not have.

5.6 Steering interventions

At each proof-line start the failure score edits the current *logits* (the scores the model assigns each possible next token). For a token j with logit ℓ_j, seen f_j times so far, we use one of two rules:

$$\text{(temperature)}\ \ \ell' = e^{-\alpha p_{\text{fail}}}\,\ell, \qquad \text{(frequency)}\ \ \ell'_j = \ell_j - \alpha\, p_{\text{fail}}\, f_j. \tag{5.1}$$

The *temperature* rule multiplies every logit by $c = e^{-\alpha p_{\text{fail}}}$ before the softmax, which is exactly sampling at a higher effective temperature:

$$\text{softmax}(c\,z) = \text{softmax}\left(\frac{z}{T_{\text{eff}}}\right), \qquad T_{\text{eff}} = \frac{1}{c} = e^{\alpha p_{\text{fail}}},$$

so the more the probe expects failure, the flatter (more exploratory) the next-token distribution. Table E.2 gives the effect at our selected α values when the probe is certain (p_{fail}=1). The *frequency* rule instead subtracts $\alpha\, p_{\text{fail}}\, f_j$ from each token's logit: a per-token penalty that grows with both the token's running count and the predicted failure probability, so once the probe senses a doomed attempt it directly discourages the tokens a repetition loop keeps re-emitting.

5.7 Selection (validation contest)

In group-by-theorem cross-validation (leakage-safe, because all line starts of a theorem stay in the same fold), the MLP and LSTM reach ROC-AUC 0.89±0.02 and 0.91±0.02 on 8B, and 0.91±0.04 and 0.92±0.02 on 32B. Throughout, ± reports the spread across folds and is not a confidence interval; we give no confidence interval on any ROC-AUC in this thesis. The contest then searches a *shared* grid, identical for both heads: {MLP,

LSTM} $\times$ {temperature, frequency} $\times\, \alpha \in \{0.15, 0.3, 0.5\}$, giving 12 steering arms plus an unsteered reference. Each arm draws K=4 attempts for each of the 90 held-out validation theorems and is ranked first by solved theorems and then by passing attempts. The same setting wins on both base models, **LSTM/frequency/**α=0.15, by +1 solved theorem on 8B (paired exact McNemar p=1.00) and +6 on 32B (p=0.146). Neither margin is significant: the contest *selects* a configuration, it does not establish that steering helps.

Table E.3 gives every arm. Two readings of it matter later. First, **the contest did not favor** $\alpha \to 0$: the unsteered base ranks fifth of thirteen arms on 8B and twelfth of thirteen on 32B, where eleven of the twelve steered arms beat it. When Chapter 6 reports that the minimizing setting in the searched range is $\alpha \to 0$, that is a statement about a different quantity, the divergence between the steered failing distribution and the succeeding one; it is neither a restatement of this ranking nor a claim about which setting proves more theorems (§6.8). Second, **the temperature rule is not the weaker option in practice**, even though at α=0.15 it moves about one point of top-token mass where the frequency rule moves 4 to 18 (§5.6): on 8B the best temperature arm ties the winner's theorem count at 41, and on 32B the LSTM temperature arm at α=0.3 reaches 44, the same as the selected arm. The two are separated by the passing-attempt tie-break, not by a gap in solved theorems, and on 32B that tie-break is fragile: the temperature arm's 115 passing attempts is a floor, because one of its attempts was still unresolved at the verification timeout.

5.8 Base architecture and hidden-state extraction

Why this prover. Three properties made Goedel-Prover-V2 the right base for this study. First, its weights are released. The probe reads the prover's internal state mid-generation and feeds its score back into that same generation, so a model we cannot open is unusable here whatever it scores. Second, it is strong. Its own report claims a new state of the art in both performance and computational efficiency, places it first overall among open-source theorem provers at its release in July–August 2025, and records its 8B model reaching 84.6% pass@32 on MiniF2F, outperforming the 671B DeepSeek-Prover-V2 despite being about 80 times smaller [23]. Those figures are the authors' own; the one we checked ourselves, its 32B MathOlympiadBench pass@32, reproduces exactly under our protocol (Chapter 6). A negative steering result on a base of this standing cannot be dismissed as an artifact of a weak prover. Third, and this is what turns the choice from a constraint into a design decision, 8B and 32B are small enough to run the whole protocol twice: 32 attempts for each of 360 theorems in a base arm and a steered arm, a hidden state captured at every proof-line start, and robust verification of every attempt. The same experiment against a 671B prover was not affordable to us.

5.9 Training data

We run the frozen base over theorems from **LeanDojo** [36] (theorems drawn from mathlib, Lean's large community mathematics library), partitioned by

theorem into a 75/25 split, then filtered to those whose reconstructed statement elaborates, which leaves 243 training and 90 validation theorems out of 1,367 and 419 (Appendix B). The validation theorems run the intervention contest (§5.7); both splits are disjoint from the test set (the 360-theorem MathOlympiadBench). For each training theorem we draw several attempts, save the hidden state at every proof-line start, and label it with the enclosing attempt's robust outcome.

Table E.4 counts what this yields. The probe learns from individual proof lines rather than whole theorems, so the small number of solved training theorems still produces a six-figure number of readings, and the imbalance the weighted loss offsets is the line-level one, roughly 93:7 failing to verifying.

5.10 Prompt

Every attempt in this thesis is generated from the completion prompt template Goedel-Prover-V2 was released with, unchanged. What differs between our two sources of theorems is only what is slotted into it. On MathOlympiadBench the prompt has *five* parts, on the mathlib-derived training and validation theorems *four*: a library lemma carries no informal statement, and its statement ends at its type rather than at `:= by sorry`. Appendix E.12 lists the parts, reports the checks we ran on both populations, and reproduces the prompt for `imo_sl_2022_N8`, one of the 360 MathOlympiadBench theorems, exactly as sent.

Two consequences matter for the rest of the thesis. The model writes its proof plan in prose and only then the Lean proof, which is why a reading taken at a proof-line start can fall in either (§5.8). We segment each generation

at the *last* Lean fence: that fence's content is the proof we splice in and verify, and everything before it is what we call the chain-of-thought. And the Lean header *is* the environment the proof is written against, so it is the environment the proof must be checked in. That is why we verify each attempt by recompiling exactly this file with the proof spliced in, and why we check in an environment matching the prompt rather than in the theorem's own file scope (§B.10), which would elaborate the proof inside a different, reconstructed scope (Appendix B).

5.11 Sampling and normalization

Sampling. For pass@k we generate K=32 independent attempts per theorem (temperature 1.0, nucleus top-p 0.95, up to 40,960 generated tokens, the identical budget for base and steered) and run each through robust verification; a theorem counts as solved when at least one of its 32 attempts verifies.

Chapter 6

Results

6.1 Setup

All comparisons use the same base model and budget: pass@32 (a theorem is solved if any of K=32 sampled proofs verifies; sampling in §5.11) on **MathOlympiadBench** [23] (360 olympiad theorems). We write *base* for the unsteered model and *steered* for the contest-selected setting (§5.7), which is **LSTM/frequency/**α=0.15 on both base models. Every number in this chapter is produced by the robust protocol (§5.1). The chapter follows the three contributions in order: the reproduction audit (§6.2); the failure signal, its transfer out of domain and what it is not (§6.3, §6.4); and the steering result with its mechanism (§6.5–§6.8).

6.2 Reproducing the Goedel-Prover-V2 baseline

Goedel-Prover-V2 reports pass@32 for its 8B and 32B models. We reproduce the *base* model on MathOlympiadBench under our robust protocol for two reasons: no per-attempt generations were released for the benchmarks it reports, so the reported scores cannot be audited attempt by attempt, and those scores come from a persistent-REPL scorer whose acceptance is systematically unsound, admitting proofs the kernel rejects (the reproducible 2=1 false positive in Chapter 5). Robust verification closes both gaps.

MathOlympiadBench. Goedel-Prover-V2 introduced MathOlympiadBench (360 olympiad theorems) and reports a 32B pass@32 of 60 (Figure 1 of [23]); no 8B figure is given. Our robust base reproduces the reported result *exactly* (Table 6.1, 60 = 60). To our knowledge our 8B number (41) is the first robustly verified evaluation of that model on this benchmark. **On this model's own output the two rules coincide.** Across all 23,040 base attempts, none rests on any of the three compiler axioms (`Lean.trustCompiler`, `Lean.ofReduceBool`, `Lean.ofReduceNat`); every other off-whitelist case is `sorryAx` or a declared placeholder, which a REPL scorer rejects as well. Goedel-Prover-V2 simply never emits the kind of proof the exploit exhibits, so our protocol and a REPL agree here, which is why the reproduction is exact. The exploit shows the agreement is not guaranteed, not that it failed on this corpus.

Table 6.1: **MathOlympiadBench pass@32**, solved of 360. "reported" reads Figure 1 of Lin et al. [23]; "robust" is our robust score of the base. "n/r": not reported.

Source	8B	32B
Goedel-Prover-V2 (reported)	n/r	60
Ours (robust base)	41	60

6.3 Probe generalization

Held-out group-by-theorem (leakage-safe) K-fold cross-validation on the 243 filtered training theorems gives ROC-AUC 0.89 (MLP) and 0.91 (LSTM) on 8B, and 0.91 and 0.92 on 32B. Transferred *out of domain* from the mathlib training theorems to MathOlympiadBench, per-line ROC-AUC is 0.83/0.83 on 8B and 0.81/0.86 on 32B (Table 6.2). Because mathlib lemmas and olympiad problems differ sharply in style and difficulty, this is genuine out-of-domain generalization: the internal states carry information about whether an attempt will later fail, and that information survives the change of domain on both base models. That the two domains really do differ for a prover is not our assumption but this model family's own measurement: adding Mathlib4 to Goedel-Prover-V1's training pool raised its ProofNet score from 13.3% to 15.6% while lowering MiniF2F from 56.6% to 54.1% and its held-out Numina set from 59.2% to 58.8%, which its authors attribute to distribution shift between the library and the competition benchmarks [22]. It is also insensitive to the label recipe: repeating the whole procedure with a weaker, rename-only labeling changes the numbers by at most a few points and does not change the ordering, so the strictness of the labeling is not what drives

separability.

Table 6.2: **Failure-prediction ROC-AUC** (0.5 chance, 1.0 perfect). In-domain is leakage-safe K-fold on the 243 filtered mathlib training theorems; transfer applies the same heads out of domain to MathOlympiadBench under robust labels. Point estimates; we report no confidence intervals.

Setting (MLP / LSTM)	8B	32B
In-domain, mathlib K-fold	0.89 / 0.91	0.91 / 0.92
Transfer → MathOlympiadBench	0.83 / 0.83	0.81 / 0.86

6.4 The signal

Two controls put the probe in context. Both are computed on the same line starts, the same attempts and the same robust labels.

Length is a strong predictor. An attempt's *total* generated length predicts its outcome with attempt-level ROC-AUC 0.92 on 8B and 0.91 on 32B; compressed length reaches 0.92 and 0.93, and the compression ratio reaches 0.88 and 0.72. That last statistic is the *DEFLATE ratio*: an attempt's compressed size divided by its raw size under DEFLATE, the algorithm behind `gzip`, which shortens text by replacing repeated phrases with back-references, so a lower ratio means more repetition.

Length is retrospective; the hidden state is not. Total length is knowable only after an attempt ends. Its online counterpart, how many tokens have been generated so far, carries no early information at all: restricted

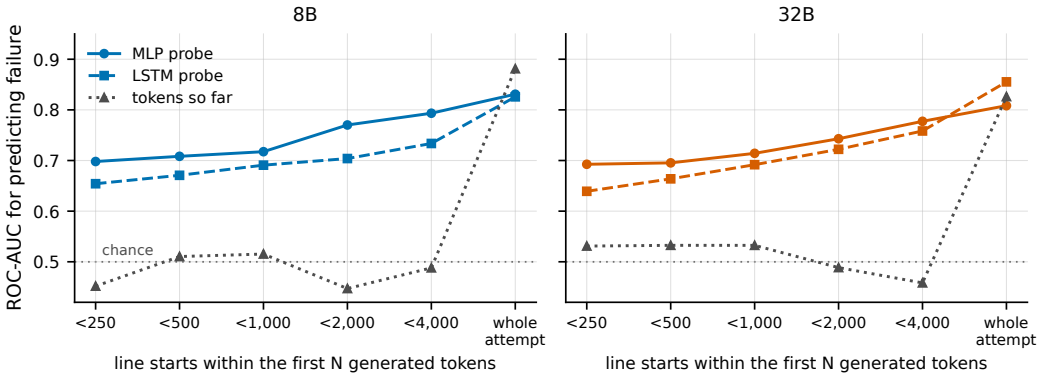


Figure 6.1: **The signal is available early; length is not.** Per-line ROC-AUC restricted to the line starts falling inside the first N generated tokens. The probes are well above chance from the first 250 tokens, while the online length feature, how many tokens have been generated so far, sits at chance until a runaway has already happened. Inside a truncated window that feature is near-degenerate by construction, so chance is its expected value; the point is that position carries no *early* information. The rightmost point is not an online comparison: over a whole attempt "tokens so far" becomes the attempt's *total* length, knowable only once it has ended, so it is a retrospective ceiling rather than a feature a controller could use.

to line starts inside the first 250 to 4,000 generated tokens it sits at chance (0.45–0.53), while the probes reach 0.64–0.79 over the same line starts (Figure 6.1). Length earns its accuracy entirely at the tail, after a runaway has already happened. The hidden state carries the signal before that, and this is the clearest evidence in this thesis that the probe is not simply a length proxy. One caveat belongs with the comparison. Inside a truncated window the token counter is near-degenerate by construction, because every line start in the window shares a narrow range of positions, so chance is its expected value rather than a surprising outcome. The claim we make is accordingly

the weaker and more precise one: position carries no *early* information, so a length rule cannot be the basis of an early intervention. We do not claim to beat a well-designed early length model, and we did not build one.

6.5 Main results

Table 6.3 reports robustly verified pass@32. **Steering does not help.** It solves 6 fewer theorems than the base on 8B (35 versus 41) and 1 fewer on 32B (59 versus 60). At k=32, the budget this comparison was designed around, neither difference is statistically significant under a paired exact McNemar test: on 8B the two policies disagree on 10 theorems, 8 favoring the base and 2 the steered policy, giving p=0.109; on 32B they disagree on 11, split 6 to 5, giving p=1.00. We therefore read the headline as a *negative* result: the hypothesis that steering helps is not supported. We use "negative" in that sense throughout the thesis, never to claim that steering is measurably worse.

The clearer effect is one level down. Counting individual verifying attempts rather than solved theorems, steering loses 99 of 560 on 8B and 153 of 897 on 32B: about a sixth of all successes on each base, in the same direction on both models and far more consistent than the theorem-level difference. This is the effect a larger study would most likely confirm first.

Table 6.3: **MathOlympiadBench pass@32, robustly verified; single-run.** Steering loses 6 theorems on 8B and 1 on 32B; neither difference is significant at k=32 (paired exact McNemar p=0.109 and p=1.00). At the attempt level the effect is steadier: verifying attempts fall from 560 to 461 on 8B and from 897 to 744 on 32B, about a sixth on each. Denominators: 360 theorems, 11,520 attempts.

	theorems solved		attempts verifying	
Method	8B	32B	8B	32B
Base	**41**	**60**	560	897
Steered	35	59	461	744
Difference	−6	−1	−17.7%	−17.1%

6.6 Validation and test disagree

The comparison that selected the intervention and the comparison that evaluates it point in opposite directions, and the contrast is worth stating on its own.

In domain, on the 90 held-out mathlib validation theorems, steering *helped*: the selected arm beat the unsteered base by +1 solved theorem on 8B and by +6 on 32B, and eleven of the twelve steered arms beat the base on 32B (Table E.3). Out of domain, on MathOlympiadBench, the same arm *lost* 6 theorems on 8B and 1 on 32B (Table 6.3).

6.7 Which theorems change

The two policies are not nested: each proves theorems the other misses. On 8B the steered policy gains 2 theorems and loses 8; on 32B it gains 5 and loses 6. Appendix F.6 names them and draws out two observations.

6.8 Why this intervention does not help

The intervention really does change generation. Under the selected setting the steered runs emit 21.5% (8B) and 14.0% (32B) fewer output bytes than the base, which is the textual signature of the repetition loop the probe was trained to detect. The frequency penalty does exactly what it was built to do. It simply does not convert into proofs, because it shortens *indiscriminately*: verifying proofs are already short (§6.4), so there is nothing to win on them, while attempts that would have converged late are truncated. That is the same effect as the sixth of verifying attempts the steered runs lose.

Three further measurements explain why no setting in our grid would have done better.

What this comparison cannot show. Two limits bound that reading. The comparison is *one step and cross-population*: it sets the steered distribution at a single line start beside the aggregate profile of attempts that succeeded, which is a different set of prefixes, selected on the outcome. What the intervention should be judged on is whether *this* attempt eventually verifies, which depends on the whole continuation, and divergence at one token is not a signed quantity with respect to that. And conditional on a prefix

the probe flags as doomed, the useful move may well be atypical of successful proofs: leaving a bad trajectory looks like increased divergence, not decreased. So a rising divergence does not establish that deviating was itself the error. What it does establish, with the strength evidence above, is that the intervention is not a nudge *toward* success-shaped generation, which is unsurprising once one notes that it is undirected: p_{fail} sets the *magnitude* of a fixed, content-blind perturbation, and neither rule carries any information about which continuation is more likely to verify. Deviations of this kind do sometimes pay, and we can count them: steering solves 2 theorems on 8B and 5 on 32B that the base never solves (§6.7). The net is negative because 8 and 6 are lost, which makes this a statement about the *rate* at which an undirected deviation helps rather than about its direction.

Chapter 7

Discussion and Conclusion

7.1 Summary of findings

We set out four questions in Chapter 4; the results answer them as follows.

Q1 (Measurement). Persistent-REPL scoring is demonstrably unsound: we exhibit a self-contained proof of `False` that a REPL scorer accepts and the kernel's axiom check rejects (§5.1). Under a robust protocol, Goedel-Prover-V2's reported 32B MathOlympiadBench pass@32 reproduces exactly (60 = 60; §6.2). This both validates the contract and shows why it is necessary. Getting the training *labels* right required equal care, because every training theorem is a library lemma that the checking environment already contains (Appendix B).

Q2 (Signal). A frozen prover's mid-proof internal state does predict failure, and it predicts it *early*. A sub-3M-parameter probe on the final-normalization hidden state reaches ROC-AUC 0.89–0.92 in leakage-safe cross-validation (§6.3), and in windows from the first 250 to the first 4,000 generated tokens it reaches 0.64–0.79 where the obvious surface feature, how many tokens have

been produced so far, is still at chance (§6.4). The information needed to tell a doomed attempt from a promising one is present in the state the model already computes, well before the attempt's length reveals anything.

Q3 (Generalization). The failure signal transfers out of domain, on both base models. Trained and selected on mathlib, the probes still discriminate failure on olympiad-level MathOlympiadBench formalizations with per-line ROC-AUC 0.81–0.86 (§6.3), and the result is insensitive to the labeling recipe. What is *not* model-independent is how useful the probes are relative to simpler alternatives: on the smaller base a linear support vector machine on a single hidden state does better than either of our heads, while on the larger base nothing we tried separates from the LSTM (§6.4).

Q4 (Steering). Using that failure probability to *smooth* the next-token distribution at each proof-line start, either by raising the sampling temperature to $e^{\alpha p_{\text{fail}}}$ or by adding a p_{fail}-scaled frequency penalty, does *not* improve the prover. Under the robust protocol, steering solves 6 fewer theorems than the base on 8B and 1 fewer on 32B, and it costs about a sixth of all verifying attempts on both base models (§6.5). This is a negative answer, and we give the mechanism behind it (§6.8): the intervention does shorten generations and cut repetition markedly, but it shortens indiscriminately and it is undirected, and the changes it makes are not a nudge toward the shape that succeeds.

7.2 Limitations

We state the limitations plainly.

- **Single-run.** Every number in this thesis comes from one run. The steer-

ing differences (-6 and -1) need repeated-run replication before any is called significant, and we do not call them so. Neither is significant at k=32, the budget the comparison was designed around.

- **Leakage-reduced labels, not leakage-free.** Our training theorems are mathlib lemmas, so the environment that checks a candidate proof already contains the target. Our labeling removes the target's own citation and its `simp` and `aesop` entries, but Lean provides no way to hide a differently-named sibling, an alias or a more general form from an imported environment; easy library goals remain over-represented; and some solved theorems are not what they are named, because the extraction span can truncate a statement or pick up a neighbouring declaration. A fourth channel sits outside the checking environment entirely and no choice of environment closes it: the base model may already carry the target's proof from pre-training, which no name-based blinding and no premise scope can reach (Appendix B.9). The headline evaluation is out of domain on olympiad theorems, where the clash cannot arise (Appendix B).

- **Small filtered training set.** Keeping only theorems whose reconstructed statement elaborates leaves 243 training and 90 validation theorems out of 1,367 and 419. The filter is safe, since a theorem with any valid proof must have a compiling statement, but the surviving set is small, so the cross-validation numbers are noisy and we report no confidence intervals on any ROC-AUC.

- **Domain gap.** Probes are trained and selected on LeanDojo/mathlib theorems and tested on olympiad-level MathOlympiadBench. This is a strength for the generalization claim but a caveat for the steering settings, which were tuned in domain. The shift is one this model family

has itself measured [22], so the reversal between validation and test is expected rather than anomalous (§6.3). The gap is not only mathematical: the prompts differ in shape too, a library prompt carrying no informal statement and ending at the statement's type (§5.10). That difference is evidently not disabling, since the base solves a far larger share of the library theorems on a far smaller budget, 88 and 92 of the 243 training theorems at eight attempts each and 40 and 38 of the 90 validation theorems at four, than of the benchmark, 41 and 60 of 360 at thirty-two. Difficulty confounds that comparison, so it bounds the concern rather than settling it.

- **The mechanism rests on a one-step, cross-population comparison.** The divergence in §6.8 sets the steered distribution at a single line start beside the aggregate profile of attempts that succeeded, a different set of prefixes, selected on the outcome. What the intervention should be judged on is whether *this* attempt eventually verifies, which depends on the whole continuation. Conditional on a prefix the probe flags as doomed, the useful move may be atypical of successful proofs, so a rising divergence does not by itself show that deviating was the error. The outcome measurement, not the divergence, is what establishes that this intervention does not help.

- **Coarse selection.** The α grid is $\{0.15, 0.3, 0.5\}$ and the contest was run once. §5.6 gives the bounds that set the grid and the three checks against the contest that place it in the right regime; they bound the regime rather than identify an optimum.

- **Timeouts and lower bounds.** Robust checking uses a 30-minute per-attempt limit, so an attempt needing longer counts as a failure and every

score is a lower bound. We believe these are close to the true values: the 32B base reproduces the reported MathOlympiadBench number exactly, and re-running timed-out attempts at a 3× longer limit recovered no new theorems.

- **Verification scope.** Robust compilation shows that a proof term type-checks under trusted axioms. It does not check that the Lean statement faithfully formalizes the natural-language problem.

7.3 Future directions

The internal-state signal is real, early, and generalizes out of domain. Four promising directions follow directly. What we tested is one point in a larger space: an undirected, p_{fail}-scaled smoothing applied at proof-line starts, with a probe calibrated out of domain. Each direction below changes one of those coordinates, which is why the negative result closes this intervention rather than the question.

Train and calibrate in domain. Our probes are fitted on 243 mathlib-derived theorems and deployed on olympiad problems, where they discriminate well but are not *calibrated*. A larger, in-domain training corpus with calibrated outputs is the single change most likely to turn this result around, and it is the one we would try first. See Appendix G.2.

A learned stopping policy. See Appendix G.2.

Steer earlier. See Appendix G.2.

A contextualized router. See Appendix G.2.

7.4 Concluding remarks

This thesis makes a narrow but, we hope, useful point twice over. First, a frozen open prover carries, in the hidden states it already computes, a signal of whether the proof it is writing will fail, and that signal is available early, transfers to an unseen problem domain, and is not merely a proxy for how long the attempt has run. Second, acting on that signal by steering the next-token distribution at each proof-line start does not make the prover better, and we can say precisely why rather than merely reporting that it did not work. A negative result of that shape is worth having: it closes a plausible line and points at the ones that remain open.

Just as important, the thesis insists that such claims be measured against the kernel rather than a server's reply, and shows, with a concrete false positive and a robust reproduction, why that insistence is not pedantry. We hope the measurement discipline outlives the specific method: any future claim of a better prover is only as trustworthy as the axioms its proofs depend on.

Bibliography

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*, 2016. URL `https://arxiv.org/abs/1610.01644`.

[2] Pawan Sasanka Ammanamanchi, Siddharth Bhat, and Stella Biderman. Faults in our formal benchmarking: Dataset defects and evaluation failures in Lean theorem proving. In *International Conference on Machine Learning (ICML)*, 2026. URL `https://arxiv.org/abs/2606.29493`. arXiv:2606.29493.

[3] Chenyang An, Zhibo Chen, Qihao Ye, Emily First, Letian Peng, Jiayun Zhang, Zihan Wang, Sorin Lerner, and Jingbo Shang. Learn from failure: Fine-tuning LLMs with trial-and-error data for intuitionistic propositional logic proving. *arXiv preprint arXiv:2404.07382*, 2024. URL `https://arxiv.org/abs/2404.07382`.

[4] Anthropic. Formalizing fermat's last theorem in lean: A timeline and selected excerpts from claude's reasoning. Technical report, Anthropic, 2026. Proof released at `https://github.com/anthropics/fermats-last-theorem`.

[5] Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it's lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. URL `https://aclanthology.org/2023.findings-emnlp.68/`.

[6] Benjamin Breen et al. Ax-Prover: A deep reasoning agentic framework for theorem proving in mathematics and quantum physics. *arXiv preprint arXiv:2510.12787*, 2025. URL `https://arxiv.org/abs/2510.12787`.

[7] Jiangjie Chen et al. Seed-Prover 1.5: Mastering undergraduate-level theorem proving via learning from experience. *arXiv preprint arXiv:2512.17260*, 2025. URL `https://arxiv.org/abs/2512.17260`.

[8] Luoxin Chen et al. Seed-Prover: Deep and broad reasoning for automated theorem proving. *arXiv preprint arXiv:2507.23726*, 2025. URL `https://arxiv.org/abs/2507.23726`.

[9] Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. Plug and play language models: A simple approach to controlled text generation. In *International Conference on Learning Representations (ICLR)*, 2020. URL `https://arxiv.org/abs/1912.02164`.

[10] Hen Davidov, Nachshon Cohen, Oren Kalinsky, Yaron Fairstein, Guy Kushilevitz, Ram Yazdi, and Patrick Rebeschini. Knowing when to quit: A principled framework for dynamic abstention in LLM reasoning. *arXiv preprint arXiv:2604.18419*, 2026.

[11] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635, 2021. doi: 10.1007/978-3-030-79876-5_37.

[12] Robert Joseph George, Suozhi Huang, Peiyang Song, and Anima Anandkumar. LeanProgress: Guiding search for neural theorem proving via proof progress prediction. *Transactions on Machine Learning Research*, 2025. URL `https://arxiv.org/abs/2502.17925`.

[13] Cezary Kaliszyk, Josef Urban, Henryk Michalewski, and Mirek Olšák. Reinforcement learning of theorem proving. In *Advances in Neural Information Processing Systems*, 2018. URL `https://arxiv.org/abs/1805.07563`.

[14] Shashank Kirtania and Arun Iyer. Steering LLMs for formal theorem proving. *arXiv preprint arXiv:2502.15507*, 2025.

[15] Matěj Kripner, Michal Šustr, and Milan Straka. LeanTree: Accelerating white-box proof search with factorized states in Lean 4. *arXiv preprint arXiv:2507.14722*, 2025. URL `https://arxiv.org/abs/2507.14722`.

[16] Sean Lamont, Christian Walder, Amir Dezfouli, Paul Montague, and Michael Norrish. 3D-Prover: Diversity driven theorem proving with determinantal point processes. *arXiv preprint arXiv:2410.11133*, 2024. URL `https://arxiv.org/abs/2410.11133`.

[17] Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurelien Rodriguez, and Timothee Lacroix. HyperTree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems*, 2022. URL `https://arxiv.org/abs/2205.11491`.

[18] Austin Letson, Leopoldo Sarra, Auguste Poiroux, Oliver Dressler, Paul Lezeau, Dhyan Aranha, Frederick Pu, Aaron Hill, Miguel Corredera Hidalgo, Julian Berman, George Tsoukalas, and Lenny Taelman. SorryDB: Can AI provers complete real-world Lean theorems? *arXiv preprint arXiv:2603.02668*, 2026. URL `https://arxiv.org/abs/2603.02668`.

[19] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL `https://arxiv.org/abs/2306.03341`.

[20] Mukai Li, Linfeng Song, Zhenwen Liang, Jiahao Xu, Shansan Gong, Qi Liu, Haitao Mi, and Dong Yu. EconProver: Towards more economical test-time scaling for automated theorem proving. *arXiv preprint arXiv:2509.12603*, 2025. URL `https://arxiv.org/abs/2509.12603`.

[21] Zhixiang Liang, Beichen Huang, Zheng Wang, and Minjia Zhang. Hidden states as early signals: Step-level trace evaluation and pruning for efficient test-time scaling. *arXiv preprint arXiv:2601.09093*, 2026.

[22] Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-Prover: A frontier model for open-source automated theorem proving. *arXiv preprint arXiv:2502.07640*, 2025. URL `https://arxiv.org/abs/2502.07640`.

[23] Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, et al. Goedel-Prover-V2: Scaling formal theorem proving with scaffolded data synthe-

sis and self-correction. *arXiv preprint arXiv:2508.03613*, 2025. URL `https://arxiv.org/abs/2508.03613`.

[24] Junqi Liu, Zihao Zhou, Zekai Zhu, Marco Dos Santos, Weikun He, Jiawei Liu, Ran Wang, Yunzhou Xie, Junqiao Zhao, Qiufeng Wang, Lihong Zhi, Jia Li, and Wenda Li. Numina-Lean-Agent: An open and general agentic reasoning system for formal mathematics. *arXiv preprint arXiv:2601.14027*, 2026. URL `https://arxiv.org/abs/2601.14027`.

[25] Numina and Kimi Team. Kimina-Prover preview: Towards large formal reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.11354*, 2025. URL `https://arxiv.org/abs/2504.11354`.

[26] OpenAI. Finite time blowup for Navier–Stokes. Technical report, OpenAI, 2026. Released September 2026, 166 pp. `https://cdn.openai.com/pdf/32d9f210-8b73-45e0-91bc-82a30aef8a9a/navier-stokes.pdf`. Establishes alternatives (C) and (D) of the Millennium problem statement. A Lean 4 formalization of both, and of a companion Euler blowup, is released at `https://github.com/openai/NavierStokesAndEuler`; its `comparator` challenge files permit exactly `propext`, `Quot.sound` and `Classical.choice`. No model is named.

[27] OpenAI. Ten advances in mathematics and theoretical computer science. Technical report, OpenAI, 2026. Released 1 August 2026; revised 6 August 2026. Lean 4 certificates at `https://github.com/openai/ten-proofs` (Apache-2.0).

[28] Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. Apollo: Automated llm and lean collaboration for advanced formal reasoning.

In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2505.05758.

[29] Z. Z. Ren et al. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025. URL https://arxiv.org/abs/2504.21801.

[30] The Harmonic Team. Aristotle: IMO-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.

[31] Niels Voss. Lean pitfalls. https://github.com/nielsvoss/lean-pitfalls, 2024. Accessed 2026-08.

[32] Evan Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, Jarod Alper, and Vasily Ilin. Learning to repair lean proofs from compiler feedback. *arXiv preprint arXiv:2602.02990*, 2026. URL https://arxiv.org/abs/2602.02990.

[33] Huajian Xin, Daya Guo, Zhihong Shao, Z. Z. Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*, 2024. URL https://arxiv.org/abs/2405.14333.

[34] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. *arXiv*

preprint arXiv:2408.08152, 2024. URL `https://arxiv.org/abs/2408.08152`.

[35] Ran Xin, Chenguang Xi, Jie Yang, Feng Chen, Hang Wu, Xia Xiao, Yifan Sun, Shen Zheng, and Kai Shen. BFS-Prover: Scalable best-first tree search for llm-based automatic theorem proving. *arXiv preprint arXiv:2502.03438*, 2025. URL `https://arxiv.org/abs/2502.03438`.

[36] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2023. URL `https://arxiv.org/abs/2306.15626`. The Python package `lean-dojo` (4.20.0, 2025) is distinct from the separate `lean-dojo-v2` project (1.0.9, 2026); the original repository also carries a tag named v2.0.0.

[37] Kevin Yang and Dan Klein. FUDGE: Controlled text generation with future discriminators. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2021. URL `https://aclanthology.org/2021.naacl-main.276/`.

[38] Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Reasoning models know when they're right: Probing hidden states for self-verification. *arXiv preprint arXiv:2504.05419*, 2025.

[39] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down

approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023. URL `https://arxiv.org/abs/2310.01405`.

Appendix A

Examples of Repetition (Goedel-Prover-V2)

In each example the base model *of that size* never proves the theorem: none of its 32 sampled attempts passes robust verification. Neither quoted attempt reaches the 40,960-token cap; they stop at 20,422 and 17,827 generated tokens, about half the budget, long after they have stopped making progress. The other size does prove both theorems (the 32B base proves `Hungary1998P6` in 3 of 32 attempts, the 8B base proves `Imo1979P5` in 6 of 32), so each attempt shows a failure of the sampling policy rather than a goal out of reach. They are instances of the common case described in Chapter 2, not of the cap-hitting minority. They illustrate the failure mode the probe is trained to detect. The second is additionally a theorem the *steered* model recovers; the first is not, and we mark that explicitly rather than implying that every degenerate failure is recoverable. Unicode characters are shown in ASCII.

A.1: degenerate lemma spam, 8B (`Hungary1998P6`). *Not recovered:* neither the base nor the steered 8B proves this theorem (0 of 32 each). The two theorems the steered 8B does recover are `Imo1960P2` and `Imo1982P3b`, each going from 0 to 2 verifying attempts of 32. The base attempt piles the same rewrite lemma into a single `simp` call 208 **times**, and an identical line recurs 106 times in the generation, without ever closing the goal:

```
simp [Finset.sum_range_succ, Finset.sum_range_succ,
      Finset.sum_range_succ, Finset.sum_range_succ,
      Finset.sum_range_succ, Finset.sum_range_succ,
      ...           -- 208 copies of the lemma in this one call
```

The pattern is the one the probe learns to recognize: the same lemma re-emitted long after it has stopped helping, with no progress on the goal.

A.2: looping `have` block, 32B (`Imo1979P5`). The base attempt cascades near-identical subgoals, each closed by `nlinarith`, which occurs 38 **times**:

```
have h12 : (a - 1) * x1 >= 0 := by
  nlinarith
have h13 : (2 * a - 8) * x2 >= 0 := by
  nlinarith
have h14 : (3 * a - 27) * x3 >= 0 := by
  nlinarith
...           -- this have/nlinarith pattern repeats 38 times
```

Recovered: the steered 32B proves this theorem, going from 0 to 3 verifying attempts of 32. It is one of the five theorems the steered 32B gains: `Usa2001P3` (0→4), `Imo1979P5` (0→3), and `Usa1979P1`, `imo_sl_2007_A1a` and `imo_sl_2019_A1` (0→1 each). `Usa2001P3` is simultaneously an 8B regression (1→0), so the two base models do not gain and lose the same the-

orems. Both examples show the failure mode the probe is meant to notice: the attempt has stopped making useful progress, yet generation continues for thousands more tokens.

Appendix B

Filtering and Target-Blinded Labeling

Our mathlib-derived theorems come from **LeanDojo** [36], a corpus drawn from mathlib, Lean's community mathematics library. That is convenient, and it is also the source of a measurement problem: the environment that checks a candidate proof already contains the theorem being proved. This appendix describes what we do about it and what remains afterwards.

Everything in this appendix applies identically to the training split and to the validation split. The same statement-elaboration filter produces both lists of kept theorems (243 training and 90 validation), and the same labeling decides both which attempts the probes are trained on and which attempts count as solved in the contest that selects the steering setting. Wherever we write "training theorems" below, the validation theorems are treated the same way.

B.1 What the probe learns from

The probe does not learn from whole theorems but from individual proof lines: at each proof-line start it takes one reading of the model's internal state and predicts whether the enclosing attempt will fail. So although only 88 (8B) and 92 (32B) library theorems are ever solved under our labeling, the probe trains on 115,285 and 177,552 line start readings, of which about 7% come from verifying attempts. Failing attempts run long, so failure readings dominate the data; training weights the rarer success readings more heavily to compensate.

B.2 Why the raw labels were unusable

Each training theorem is reconstructed under its original library name. Under `import Mathlib` that name already exists, so Lean rejects the reconstructed declaration as *already declared* before the proof body is ever checked, and records a failure even when the model's proof is correct. This force-failed roughly three quarters of the training theorems (1,018 of 1,367), recording correct proofs as failures.

B.3 Filtering: keep statements that elaborate

Fixing the clash exposed a second problem. We use Goedel-Prover-V2's released prompt and elaborate each proof under the context that prompt shows the model, so we never supply file-local context the model has not seen. Reconstruction slices a theorem out of its source file but drops the sur-

rounding `variable` declarations, instances and `open` commands, so a statement that needs that context does not elaborate at all, independently of any proof: the errors are `failed to synthesize`, `function expected` and `unknown identifier` on a section variable. We therefore compile `header + statement := by sorry` for every theorem and drop those that do not elaborate.

The filter is safe by construction. A theorem with any valid proof must have an elaborating statement, so dropping non-elaborating theorems *cannot* remove a positive; what it removes are theorems on which every attempt is a forced negative, which would otherwise teach the probe reconstruction noise rather than proof failure. It also preserves consistency between training and deployment, since we do not add in-file context that would make the verified statement differ from the one the states were captured against. Table B.1 gives the counts.

Table B.1: **Theorem filtering.** Stage 1 drops non-colliders whose statement does not elaborate. Stage 2 repeats the test with the rename applied, which exposes colliders whose statement-elaboration failure the name clash had been masking: the clash fired before Lean ever checked the statement.

Stage	train	validation
Initial split	1,367	419
After stage 1 (dead statements)	1,080	319
After stage 2 (statement elaboration)	**243**	**90**

B.4 Target-blinded labeling

A proof should stand without the theorem it is proving. At checking time we therefore blind the environment to the target lemma: we give the reconstructed head a fresh name, switch off the target's own automation entries, and discard any proof that names it. The blinding is partial, and Section B.9 says exactly what it cannot reach.

On the surviving theorems we assign each attempt its label by compiling its stored proof under a header applied *only at checking time*, never in the prompt the model saw. Four mechanisms, in order:

1. a **rename**, giving the reconstructed head a unique suffix so the declaration is legal;
2. `attribute [-simp] X`, removing the target lemma's own `simp` entry, emitted on every collider, that is, wherever the target actually exists in the library. It closes the route a bare `simp` takes through that entry *where the target has one*, and reaches neither a differently-named sibling nor the target supplied explicitly;
3. `attribute [-aesop] X` in addition, but only where the target is a registered `aesop` rule. This one is gated because it *errors* when no such registration exists, where the `simp` command only warns;
4. a **citation filter** that drops any surviving success whose proof text names the target lemma in a citation position. This is what catches the explicit forms the erases cannot reach: `simp [X]`, `exact X`, `apply X`, and the `aesop add` clauses.

The division of labour is worth stating plainly: the erases handle the *bare*

tactic, and the citation filter handles the *explicit* form. Neither erase removes the theorem, only its registrations, so X stays available to any proof willing to name it, and closing that is exactly what the filter is for. We then additionally drop attempts that fail with the guard active but pass without it, since those depended precisely on the crutch we removed. A positive therefore means: compiled under the guard, without citing the target, and not dependent on the target's own automation entry.

B.5 What counts as a label

The rules above can be stated exactly. For an attempt a, one theorem paired with one sampling seed, verification yields four predicates:

$h(a)$ the attempt compiles under the checking header of §B.4 (the `__us` rename, `[-simp]` on colliders and the conditional `[-aesop]`) and passes robust verification;

$s(a)$ it compiles under the rename alone, that is with the target lemma still fully available, both by name and through its own automation entries;

$c(a)$ its proof text names the target lemma;

$e(a)$ the prepended attribute line itself errored, which the attribute-error guard detects so that a broken checking header is not counted as a failed proof (Section B.8).

Writing A for the attempts of a split, the label sets are

$$P_{\text{train}} = P_{\text{val}} = \{a \in A : h \wedge \neg c\}, \qquad N_{\text{train}} = \{a \in A : \neg s \wedge \neg e\},$$

$$D_{\text{train}} = \{a \in A : \underbrace{h \wedge c}_{\text{citation filter}} \vee \underbrace{s \wedge \neg h}_{\text{automation erase}} \vee \underbrace{e}_{\text{attribute guard}} \},$$

$$A = P_{\text{train}} \sqcup N_{\text{train}} \sqcup D_{\text{train}}.$$

In words, a positive is an attempt the kernel accepts with the target's own automation entries removed and without naming it, and a *negative label* is one the kernel rejects *even when the target is fully available*, by name or through its automation. The three terms of D_{train} are exactly the three attempt-level mechanisms, so the partition is also the reason each of them exists: each mechanism is why one class of attempt is neither a positive nor a negative label.

Two implications hold, and we measured both rather than assuming them, over all 3,888 training attempts. First, $h \Rightarrow s$ with no exceptions: the checking header only ever *removes* lemmas from the environment, so anything compiling under it compiles without it, and the two compiles nest. This is worth checking rather than assuming, because dropping a `simp` lemma can in principle stop `simp` diverging or reaching a dead normal form; it simply never does here. It is also why the negative rule can be written $\neg s$ rather than $\neg h \wedge \neg s$. Second, $e \Rightarrow \neg h$, since the guard is consulted only on a failure, so $\neg e$ excludes only an attribute error whose proof also fails unguarded. No training attempt is of that kind, but the term is not redundant and we keep it.

Validation applies the same positive rule, $P_{\text{val}} = P_{\text{train}}$, and differs in two ways. Nothing is discarded there, so there is no D_{val}: a citing pass or an attribute error is simply not a solve rather than being removed. And **its negatives are never used**, because the contest ranks arms by theorems solved and then by passing attempts, both of which count positives only.

What the mechanisms cost. Of the attempts the kernel accepts with the target fully available (458 on 8B and 470 on 32B), the automation erase

removes 23 and 2 (5.0% and 0.4%) and the citation filter 71 and 51 (15.5% and 10.9%), leaving the 364 and 417 positives the probes train on. Among the negative labels, 3.8% (8B) and 4.2% (32B) name the target, against 16.3% and 10.9% of accepted proofs, so citation is roughly four times and two and a half times more common among proofs that pass than among those that fail. How many failures reach the target *through automation* is very difficult to measure, because a failed attempt has no completed proof to attribute steps to.

Both definitions are the strictest available in this setup. On the positive side that means the smallest set: every alternative, dropping the citation filter or the erase as well, is a superset. On the negative side it means the highest evidential bar, an attempt counting as a failure only if it fails with the target fully available; the laxer alternative $\neg h$ would admit attempts that fail only once the guard is applied. Softening either definition changes little: relaxing the positive rule adds at most 94 attempts on 8B (+25.8%) and 53 on 32B (+12.7%), and relaxing the negative rule, by no longer counting a failure that names the target, removes 56 (−3.8%) and 62 (−4.2%). We keep those 56 and 62 deliberately: an attempt that was given the target and still failed is the best-evidenced negative there is.

Of the 333 retained theorems, 325 collide with an existing library lemma and 8 do not; of the colliders only 2 are registered `aesop` rules, so mechanism 3 is a correctness guard rather than a driver of the numbers. Under this labeling 88 (8B) and 92 (32B) of the 243 training theorems have at least one positive attempt, 103 counting either base.

B.6 Why each guard is needed

A tested example makes the dependency concrete. Take X = `List.reverse_reverse`, the `@[simp]` lemma l.reverse.reverse = l. The renamed copy `theorem reverse_reverse__us …:= by simp` still *passes*, because bare `simp` closes the goal through X's own `@[simp]` entry: renaming alone is not enough. Adding `attribute [-simp]` X makes that same proof *fail*, so the erase is necessary. But `…:= by simp [`X`]` re-adds X explicitly and passes again, and is caught only because the citation filter sees the named X. All three files compile under the robust verifier.

B.7 Why the citation filter matches positions, not mentions

The filter asks *where* the target's name appears, not whether it appears: after `exact`/`apply`/`refine`, immediately after `:=`, after `using`, before a rewrite arrow, or inside a `[…]` bracket. The obvious alternative, dropping any success whose proof contains the name at all, is worse in both directions. We measured both over the positive labels of the training and validation theorem proof attempts, 781 in training and 2,617 in validation, 3,398 in all, which is exactly the set the deployed filter passed. Table B.2 gives both.

The two failure modes are opposite, which is the reason for the design. The 71 positives a short-name rule would discard are not citations at all: Lean's identity function `id` in $g \circ f$ = id (64 attempts, target `Function.LeftInverse.id`), the `ext` *tactic* (3, target `Group.ext`), a `.map` field access (3, target `Matrix.IsDiag.map`), and one `simpa [toReal]`

Table B.2: Two naive alternatives to the citation filter. The first column is measured on the 3,398 positive labels, the second on the 122 citations the deployed filter removed.

rule	positives it would discard	citations it would miss
deployed: the name in a citation position	*n/a* (this set is its output)	*n/a*
naive: **short** name anywhere	71 of 3,398 (2.1%)	0
naive: **full** name anywhere	0	29 of 122

`using Real.toReal_zero` (1, target `EReal.toReal_zero`). That last one is not a lemma from another namespace, which is what we first assumed: `Real.toReal_zero` does not exist, and the attempt compiles because the `simp` call closes the goal on its own, with Lean's linter suggesting `simp` in place of `simpa`. The statement is true by definition, so no lemma is involved at all. A full-name rule never over-catches, but of the 122 citations the filter did remove from the accepted training attempts (71 on 8B and 51 on 32B, the counts reported above), only 93 carry the full name: the other 29 name the target by its short name alone, and a full-name rule would have admitted all 29 as positives. The short name is required, and requiring a citation position is what makes matching it safe.

A name rule would also fire on the model quoting the theorem it was asked to prove. Blanking Lean comments, `--` lines and nestable `/- -/` blocks, and re-checking where the name really sits, the target's full name appears in 373 attempts and in 29 of them *only inside a comment*, and the *short* name, which is what the filter actually matches, is comment-only in 149, five times as often. None of those 178 is on an attempt that compiled under the guard,

where the citation filter is never consulted, so nothing would have changed here; nothing prevents the same echo in a positive, and there it would discard a genuine success for quoting the statement it was given.

We also checked the opposite risk directly. Sweeping all 13,248 training and validation attempts (243 theorems $\times$ 8 attempts $\times$ 2 base models in training, $90 \times 4 \times 13$ contest arms $\times 2$ in validation) for a target name appearing anywhere the filter does not look, 350 attempts mention it uncaught. Of those, 33 use the full name, all in *failing* attempts, and 29 of the 33 are the model echoing the theorem statement in a comment rather than citing it. The other four put the name in code: one restates the theorem as a nested declaration, and three are genuine citations of the form `have h := @X …`, which the filter misses because Lean's explicit-argument prefix `@` defeats the `:=` and `using` patterns, while the `exact`/`apply`/`refine` and bracket patterns allow arbitrary text and survive it. The remaining accepted mentions are the short-name false alarms above. No accepted proof cites the target through a form the filter missed.

Those three missed citations are not leakage escaping, and the label definitions are what settle it. The filter is consulted *only* on successes, since the positive rule is $h \wedge \neg c$; a citing failure keeps its negative label. The negative rule is $\neg s \wedge \neg e$, with no c term at all, so a negative is by definition an attempt that failed *with the target fully available*, by name and through its automation, and permitting that is exactly what makes the negative rule strict. Those three are that rule working: the model applied the very lemma it was asked to prove and the kernel still rejected the proof. The `@` gap would matter for an *accepted* attempt of that shape, and none occurs. Like `aesop (add norm simp X)`, it is left unfixed, since the labels are frozen.

Two limitations of the filter, both measured. The first is that it reads the raw proof text. Lean has four comment forms, `--` to end of line, the nestable `/- -/`, and the doc forms `/-- -/` and `/-! -/`, and the filter strips none of them, so a proof quoting the target in a comment can be flagged. Blanking all four, the full name sits only inside a comment in 29 attempts (12 in training, 17 in validation) and the short name in 149 (44 and 105), and **not one of those** 178 **is on an attempt that compiled under the guard**. The filter is consulted only on successes, so it was never asked about any of them: this is stronger than saying no positive was affected, and it holds in both splits. The opposite direction is measured on the same population: the filter fires on 591 attempts, and every one still fires once all four comment forms are blanked, so no drop is caused by a comment either. The firings on attempts that did compile under the guard number 122 in training and 206 in validation, which are exactly the drop counts above. A stripping pass is still the first thing to add in a future run, since neither bound is guaranteed by the design; it simply cost nothing here.

The second is subtler. The filter matches the target's *short* name, and a different lemma sharing that name can be in scope through the header's `open` or through the namespace itself. Call it a *short-name homonym*. Of the 122 drops, 29 matched on the short name alone, and resolving each cited identifier in its own scope with Lean shows 22 really are the target and 7 are homonyms. Whether that is a mistake depends on what the homonym says. Four cite `mul_comm`, the general `CommMagma` lemma, where the target is `EReal.mul_comm`: that is the target's own *more general form*, which instantiates to it. One cites `Complex.abs_re_le_abs`, which states exactly the target's statement under another name, an *alias*. Both are leakage channels listed in Section B.9, so five of the seven are the filter catching something

real by an accident of naming. The remaining two cite a genuinely different lemma. These are therefore *potential* false drops: restoring all seven would make the theorem counts 89, 94 and 105 rather than 88, 92 and 103, and would touch no benchmark number, since MathOlympiadBench involves no labeling. The labels are frozen, so we record this rather than correct it.

The `EReal.mul_comm` case shows why no regular expression settles this: inside `namespace EReal` the bare `mul_comm` denotes the general lemma, because the target is `protected` and does not shadow the root name. Deciding it needs Lean's own name resolution.

B.8 The attribute-error guard, and what it does not catch

The guard behind $e(a)$ exists because our own header can be the thing that fails. It fires only on a collider, only on an attempt that already failed, and only when a Lean error names the target as a *quoted* identifier together with one of `not registered`, `unknown identifier` or `unknown constant`. Table B.3 gives the three cases that matter.

Two weaknesses follow, and they have different causes. The **message list** omits `ambiguous identifier`. The **quoting** is defeated by a prime: Lean writes an identifier as `'X'`, so a sibling `X'` makes the target's own quoted form a substring of an error that has nothing to do with the header. We leave both as they ran, because the labels are frozen and either fix would change what was measured; adding the missing message and anchoring the quoted match at a word boundary would close them.

The consequence is worth stating plainly rather than leaving implicit: **the**

Table B.3: The attribute-error guard on the cases that arose, and on the case it is for.

	theorem	what happened
where it fired	`Ordinal.lift_lt`	fired, and it was a **false positive**. The error came from the *body*: the proof used the primed sibling `Ordinal.lift_lt'` and applied `.mpr`, which Lean reports as `unknown constant 'Ordinal.lift_lt'.mpr'`. The attribute line compiled cleanly
what it missed	`gcd_greatest`	the attribute line genuinely failed, with `ambiguous identifier`, which is not one of the three matched messages, so the guard stayed silent
where it would work	any absent target	`unknown constant 'X'` on the attribute line: the guard fires and the attempt leaves every label set

guard never bound. It fired zero times on both bases in training, and once in validation on an attempt that was failing on its own merits, so it changed no label and no count anywhere in this work. Its single blind-spot case, `gcd_greatest`, is also the *only* broken header in the whole project, which we measured rather than argued: no training theorem shows an error on its attribute line in the stored compiles, and since a broken header fails every attempt of its theorem, we compiled the header of each validation theorem that scored zero in all 13 contest arms, the only ones that could hide one, and found this one.

B.9 What still leaks

These guards remove *direct* dependence on the target and nothing more. Four channels remain, and the last one is outside the checking environment altogether:

- The target can be reached through a differently-named `@[simp]` or `@[aesop]` sibling, or a more general form. Lean provides no way to remove a sibling, or indeed any declaration, from an imported environment: only name *resolution* can be hidden, not availability, and we cannot enumerate the equivalents.
- The target can be reached under an alias that a name-based filter does not match.
- An easy goal can be closed using other library lemmas, or be true by definition and need no lemma at all, which is valid but is not the olympiad-style reasoning we test on.
- **The base model may already carry the target's proof.** Our theorems are mathlib lemmas and mathlib is public Lean source, so memorization is a leakage route that no choice of environment closes. A memorized proof is not reached by name resolution, so neither the rename, the two attribute erases, the citation filter, nor the pre-theorem scope of §B.10 touches it; it is the one channel a different checking environment cannot help with. We can bound what is documented but not exclude it, and the bounding has to follow the chain of *weights* rather than the family tree (§D.1). No documented supervised stage in Goedel-Prover-V2's ancestry trains on a curated mathlib proof corpus. The two models in the wider family that do, DeepSeek-Prover-V1.5 in its supervised set [34]

and Goedel-Prover-V1 with its added Mathlib4 examples [22], are sibling branches whose weights never reach our base, so neither is evidence here. What cannot be excluded is pre-training: DeepSeek-Prover-V1.5-Base, which *is* in the ancestry, was pre-trained on formal-language corpora including Lean [34], Qwen3's corpus is undisclosed, and our base is supervised on text DeepSeek-Prover-V2 generated, so anything memorized upstream can travel as generated text. We state this channel rather than measure it, and it is a further reason the headline evaluation is out of domain on theorems that are not library lemmas.

Proving each theorem with only its pre-theorem dependencies in scope does remove the target by construction, and it is implemented rather than hypothetical; Section B.10 weighs that alternative against ours. We therefore describe our labels as *leakage-reduced, not leakage-free*, and we make the headline evaluation out of domain on MathOlympiadBench, whose theorems are not library lemmas, so the clash cannot arise there at all.

B.10 Prompt and elaboration environment: four options

Our statements come from LeanDojo and are fitted into Goedel-Prover-V2's released prompt, so the options below are not rivals over where a statement comes from. They differ in what a candidate proof is *checked against*, and the axis that matters is whether the model was shown the context it is judged against. Table B.4 sets them side by side.

Rows one and two break the match. In both, a proof is checked against context the prompt never carried. Row one is the inspected `lean_dojo_v2` package, and its two columns come from two different code paths, which is worth keeping apart. Its *whole-proof* path hands the model the extracted theorem statement verbatim, a source slice, which is why that statement can be fitted into a released prompt at all and is the part we reuse. Its *search* path supplies the elaboration environment: it imports the target module and starts the goal from the theorem's printed elaborated type, which its own documentation notes is a different operation from resuming Lean at the original proof position, and its bundled whole-proof path does not verify a candidate at all. The split matters for leakage: only the search path reaches an accept-or-reject, so only it can return a verdict contaminated by the target it keeps in scope, while the whole-proof path returns no verdict to contaminate. Row two is the idealised version that row one is often assumed to be: compile the theorem's own file with only its proof replaced, and exactly the preceding declarations are in scope. Row two is therefore the only option that excludes the target and its consequences *by construction*, and it needs no tracing machinery, only the repository revision, the toolchain and one proof substitution. Yet the mismatch survives it: section `variable`s, `include`d hypotheses, earlier definitions and local notation are all live at checking time and none of them appears in the prompt. The objection is therefore structural rather than an artifact of one implementation, which is why we state it against row two and not only against row one.

It is worth naming where the assumption behind row one comes from, because we made the error ourselves before auditing the package. LeanDojo's traced *dataset* records, for each theorem, the premises accessible before it, which is what a retrieval model is trained on. That is a different artifact from the

v2 prover's checking path: the search path takes the theorem's printed type from an environment in which the target's own module has been imported, so the target is available rather than removed. A premise-scoped environment is implemented, but by `lean_dojo.check_proof` in the *original* package, not by `lean_dojo_v2`.

Why not the full-context prompt. Row four closes the gap honestly, and it is the workflow the `lean_dojo_v2` documentation recommends: put the file prefix in the prompt and verify at the original location. The cost is the prompt itself. Goedel-Prover-V2 is decoded from a fixed completion prompt: an instruction to complete a Lean 4 file, the Lean header, an informal statement of the problem where the theorem has one, the formal theorem definition, and a closing instruction to plan the proof before writing it (§5.10). Replacing that with a file prefix carries a great deal of material irrelevant to the theorem at hand, other declarations and unrelated local context, and moves the model off the distribution it was released and evaluated on. Our base scores are compared against the released model's own, so that shift would put the comparison in question.

What we do instead, and what it costs. We keep the released prompt structure and make the elaboration environment match it: the exact statement, the canonical header, the theorem's namespace, and the blinding and filtering of §B.4. A statement that needs local context the prompt does not carry simply fails to elaborate, and the filter drops it before it can become a label. That is the price of the choice, and it is large: about 80% of the candidate theorems go. A proof is then checked in the environment the model was actually shown, and a failure means the proof failed rather than that the

checker knew something the prompt withheld.

The alternative, and why we did not take it. Proving each theorem with only its pre-theorem dependencies in scope does remove the target by construction, and it is implemented rather than hypothetical: `lean_dojo.check_proof` replays the file prefix, splices the candidate proof in and recompiles, and has done since version 4.20.0, released in June 2025. We did not use it, and the reason is a trade-off rather than an oversight; Section B.10 sets the options side by side. Under that replay the target is unreachable at its own position, so the rename, both attribute erases and the citation filter all become unnecessary. The price is that the proof is elaborated under the file's *own* imports, which are narrower than the canonical header the model was prompted with, so a correct proof can fail for using a lemma or tactic from elsewhere in Mathlib, a failure that is not mathematical. We keep the elaboration context matched to the prompt and pay for it in leakage; the replay lowers leakage and pays for it in a mismatch of the other kind. Neither dominates.

That the header matters is measurable rather than stylistic. The population that matters is the one the checking environment applies to, the 3,398 positives on the retained training and validation theorems, 781 in training and 2,617 in validation. Counting those that use a tactic, notation or unqualified name the canonical header supplies, with the benchmark's verifying attempts beside them for comparison:

the header supplies	positives (3,398)	benchmark (1,457)
`aesop`, which needs `import Aesop`	762 (22.4%)	184 (12.6%)
`norm_num` / `positivity` / `polyrith` / `gcongr`	600 (17.7%)	1,092 (74.9%)
big-operator notation	130 (3.8%)	109 (7.5%)
an unqualified name reachable through the `open`s	27 (0.8%)	137 (9.4%)
any of these	**1,398 (41.1%)**	1,188 (81.5%)

Two in five of the proofs our labels are built from lean on something the header provides, so an environment that does not supply it rejects correct proofs. The mix differs by domain, which is why both columns are shown: `aesop` leads on the mathlib theorems and `norm_num` on the olympiad problems, and the unqualified-name channel nearly vanishes on the library theorems, 0.8% against 9.4%, because those files sit in their own namespaces and the model writes qualified names. These are textual counts over proof bodies rather than an elaboration-level dependency analysis, so each figure bounds from above what would break without the header; neither is a measured failure rate.

The two *environments* cannot be merged in one file. Injecting `import Mathlib` into a replayed prefix re-imports the finished target, so the by-construction removal is lost whatever the prefix is renamed to, and the public declarations in the prefix collide with their imported copies. We checked this in LeanDojo's own environment, which is the fairest available test because the control and the hybrid are then the same LeanDojo code with one parameter changed. Its interactive session enters the replayed prefix and returns the theorem's goal; adding `import Mathlib` to the same

construction makes it fail, and the file LeanDojo generates then carries 286 *already declared* errors, the first of them on the target itself. We ran this on mathlib because the alternative repositories do not depend on Mathlib, so injecting it there would report an unknown module rather than a collision. Repairing that by renaming would additionally require resolving every later reference in the prefix and in the retained suffix. What we leave to future work is therefore pairing premise-scoped checking with our axiom protocol, not the replay itself: as its source shows, `check_proof` accepts on compilation together with a `sorry`-warning match and never inspects the axiom set, so it answers a weaker question than Section 5.1 does. Run on a repository of its own, it accepts a proof whose axiom set contains `Lean.ofReduceBool`, which our whitelist excludes, so a proof it accepts can be one we reject. We have not run it over our attempts, so we make no claim about what it would do to our labels.

Premise-scoped checking and our labeling remain complementary rather than ordered, and it is worth being exact about what each excludes, because neither is leakage-free. A pre-theorem scope puts the target and everything declared after it out of scope, which our name-based blinding cannot reach; `check_proof` retains the remainder of the file verbatim and compiles it, so what changes is what the proof can cite at its own position, not what the file contains. It does *not* exclude a more general form or a differently-named sibling declared *earlier* in the dependency order, and that is the common case: a generalization is normally proved before the specializations that use it. Compiling the prefix of `EReal.mul_comm` up to its own position and asking for the general `CommMagma` lemma, `mul_comm` still resolves; only the missing `CommMagma EReal` instance, established later in that same file, stops it applying there. A mathlib-derived corpus therefore carries some leakage under

any environment, and the two leakage controls are worth applying together rather than choosing between (§B.9). The controls compose even though the environments cannot: name-based blinding is applied inside whichever environment is chosen.

Table B.4: **Four ways to pair a prompt with an elaboration environment.** Only the last two show the model the context its proof is judged against. We take the third.

	Prompt	Elaboration environment	Sees it?
`lean_dojo_v2`	the extracted statement, a source slice, which is what its whole-proof path hands the model; fits into the released prompt	its *search* path imports the target module and starts from the theorem's *elaborated type*, so section `variable`s and instance arguments enter the goal, the target stays available, and notation, options and `open`s are not restored; the bundled whole-proof path does not verify	No
Pre-theorem file scope (`lean_dojo.check_proof`)	the same statement, same prompt	the theorem's file with only its proof replaced, recompiled; exactly the preceding declarations are in scope, so this *does* remove the target by construction, but section variables and local notation are present and were never shown	No
Ours	the released prompt	canonical header, exact statement, namespace, blinding and filtering; a statement needing context the prompt lacks fails to elaborate and is dropped, which costs about 80% of candidate theorems	**Yes**
Full-context prompt	the file prefix up to the proof	the same file context; prompt and environment agree, but the prompt carries much irrelevant material and leaves the distribution the model was released on	Yes

Appendix C

Positioning Against the Compute-Heavy Frontier

The strongest current formal provers use vastly larger parameter, search and interaction budgets than we do, and the gap has widened sharply over the course of this work. Reinforcement-trained whole-proof and lemma-style provers scale test-time sampling, refinement and conjecture pools [8, 23, 25, 29]; agentic frameworks then wrap general-purpose closed models in tool-using Lean loops [6, 7, 24].

Three more recent results put the scale gap beyond argument. Aristotle reached gold-medal-equivalent performance on the 2025 International Mathematical Olympiad, producing formal Lean solutions to five of the six problems [30]. In August 2026 a team of Claude agents formalized *Fermat's Last Theorem* in Lean over eleven largely autonomous days, producing 29,511 theorems and roughly 13 million lines of Lean on top of Mathlib, with humans writing no mathematics and no Lean beyond the one-line statement of the goal theorem [4]. In the same month OpenAI released ten results on problems

in mathematics and theoretical computer science that had resisted progress for a decade or more, each obtained by an internal model the manuscript does not name and shipped with a Lean 4 certificate [27]; the released formalizations run to roughly 548,000 lines of Lean. In September 2026 the same group released a 166-page construction of a finite-time blowup for the three-dimensional incompressible Navier–Stokes equations, establishing alternative (C) of the Millennium problem statement as posed by Fefferman, and alternative (D) on the torus [26]. That manuscript names no model, but a Lean 4 formalization of both alternatives, and of a companion Euler blowup, was released alongside it.

Table C.1 contrasts those settings with ours: a frozen open base, a low sample budget, one trained probe, no base-model reinforcement learning, no tools, and no agentic loop. The comparison is one of *budget*, not of quality, and we make no absolute-success or state-of-the-art claim. The same probe could in principle steer generation *inside* such an agent; here we test the narrower low-budget case.

C.1 The measurement contract, independently adopted

The last four are worth singling out for a reason unrelated to scale: all four adopt the acceptance rule this thesis argues for, and two publish it as a machine-readable artifact.

Aristotle counts a problem solved only given "a complete proof using the Lean 4 proof language and its mathematical library Mathlib, without gaps or unsound axioms like `sorryAx`" [30]. The Fermat's Last Theorem

proof is reported as resting on Lean's three standard axioms with no `sorry`, and was rechecked by two independent external checkers [4]. The ten-advances release goes further and records, for *each* of its ten results, a declared `sorry_count` of 0 together with the axiom list `[propext, Classical.choice, Quot.sound]`, and ships a per-result configuration for an independent checker [27]. The Navier–Stokes and Euler formalization, released days before this writing, declares the same three axioms in the configuration files it ships for the `comparator` external checker, and is the largest single claim of the four.

That axiom list is, verbatim, the whitelist of §5.1. Because both releases are public, we did not have to take either claim on trust: we downloaded them and confirmed directly that all ten of the ten-advances files, some 548,000 lines of Lean, and the 2,657 files and 640,860 lines of the Navier–Stokes and Euler development, contain no `sorry` and declare no axiom of their own. The only four `sorry`s in the latter sit in its `ComparatorChallenges` directory and are the challenge *statements*, the formalized Clay problem the solution modules are checked against, which is what they are for. Doing that check rather than believing the accompanying report is precisely the discipline this thesis argues for, applied to someone else's result.

At the scale of a machine-written development of tens of thousands of theorems, checking the axiom set rather than trusting the prover's own report stops being a methodological nicety and becomes the only thing that makes the result mean anything. That is the same argument this thesis makes at a far smaller scale, and the convergence is, we think, evidence that the measurement contract rather than the particular steering method is the part of this work most likely to outlast it. We note that none of these results has yet been through peer review, and that no Millennium Prize has been awarded

for the Navier–Stokes one: the Clay conditions turn on refereed publication and sustained acceptance, and the work is days old.

Table C.1: Representative Lean provers. Parameter counts are exact where they are public; where they are not, † marks a model with no published count and ‡ a proprietary model whose size is undisclosed; "MoE" = mixture-of-experts (only a fraction active per token). Frontier and agentic rows quote external reports; our row gives exact probe counts and keeps the base fixed.

System	Params	Model architecture	Inference budget
Specialized reinforcement-trained provers			
DeepSeek-Prover-V2 [29]	671B MoE	DeepSeek-V3 MoE	pass@8192
Kimina-Prover [25]	72B	Qwen2.5-72B dense	pass@8192
Seed-Prover [8]	†	Seed family	Light 64–256; Heavy days
Seed-Prover 1.5 [7]	†	Doubao-Seed	pass@8 × 8; ≤ 28 tool calls
Agentic frameworks and frontier results over general-purpose (closed) models			
Numina-Lean-Agent [24]	‡	Claude Opus 4.5	agentic, many iterations
Ax-Prover [6]	‡	Claude Sonnet 4/4.5	≤ 400 API calls; pass@1
Aristotle [30]	†	Lean search + informal reasoning	IMO-scale search
Claude on FLT [4]	‡	multi-agent Claude	11 days, 29,511 theorems
Ten Advances [27]	‡	OpenAI, unnamed internal model	10 open problems, 548k Lean lines
Navier–Stokes blowup [26]	‡	OpenAI, model not stated	Millennium alternatives (C) and (D); Lean certificate
Frozen open base model + trained probe (this work)			
Goedel-Prover-V2 [23]	8.19B / 32.76B	Qwen3 dense	pass@32
This work	0.67/0.81**M (LSTM, deployed)**	**MLP / LSTM-128**	**pass@32**

Appendix D

Supplementary Background, Scope, and Provenance

D.1 How the base prover was trained

Every result in this thesis sits on top of a model we did not train, so it is worth setting out briefly how that model was produced and on what. Nothing in this section is a result of ours; it is read from the four papers that describe the line of descent [22, 23, 29, 33, 34].

Five stages, with the compiler as the only teacher. Goedel-Prover-V2 starts from Qwen3-8B and Qwen3-32B and is trained in five stages [23]. It *collects* proofs by having DeepSeek-Prover-V2 generate at scale and keeping only what Lean accepts; *imitates* them by supervised fine-tuning, which is the step that turns a general model into a prover; learns to *repair* by training on a failed proof together with the compiler's error and the correction; *scaffolds* new problems by turning failures into easier sub-goals and successes

into harder variants; and finally *optimizes* with one epoch of reinforcement learning whose reward is simply whether Lean accepts the proof. A proof that does not compile is never trained on, and the compiler is never itself trained. After every stage the weights are averaged back toward the base, $\theta = (1 - a)\,\theta_{\text{base}} + a\,\theta_{\text{trained}}$, on the stated ground that training raises pass@1 while lowering pass@N and a prover that samples many attempts lives on that variety.

Where its statements come from. There is no counterpart to our LeanDojo corpus. The paper names no source library: its statements are natural-language competition problems translated into Lean by its own formalizer, plus problems manufactured by the scaffolding above. That is true of the whole family, and it is the sharpest contrast with our own training set, which is fixed, public and mathlib-derived, so that we can filter and audit it one theorem at a time (§5.9, Appendix B).

The chain of weights. The distinction between this chain and the wider family tree matters later (Appendix B.9), so we give it explicitly.

model	starts from	where its statements come from
DeepSeek-Prover [33]	DeepSeekMath 7B	869,659 scraped competition problems
DeepSeek-Prover-V1.5 [34]	the same, pre-trained on Lean	Mathlib4, Lean Workbook, V1's synthetic set
DeepSeek-Prover-V2 [29]	DeepSeek-V3 671B	Lean Workbook, Goedel-Prover-V1, its own sub-goals
Goedel-Prover-V1 [22]	DeepSeek-Prover-V1.5	Numina and AOPS, via two trained formalizers
Goedel-Prover-V2 [23]	Qwen3 8B and 32B	its own formalizer plus manufactured problems

Two things in that table are easy to misread. Goedel-Prover-V1 is *not* an ancestor of Goedel-Prover-V2's weights, and neither is the supervised version of DeepSeek-Prover-V1.5, since DeepSeek-Prover-V2's smaller model starts from the *base* checkpoint. So the two places where a curated Mathlib4 corpus is explicitly trained on sit in sibling branches. And the line of descent is circular, because DeepSeek-Prover-V2 draws training problems from Goedel-Prover-V1 in turn.

Two facts the rest of the thesis uses. First, **the recursive sub-goal splitting is a trained objective rather than an emergent habit**. DeepSeek-Prover-V2, the model ours is distilled from, was cold-started on proof skeletons made of `have` sub-goals that a smaller prover then discharged one at a time; those sub-goals were fed back as fresh training problems; and its reinforcement learning added a *consistency reward* penalizing any proof that dropped one of the decomposed lemmas [29]. That is why §2.1 can say the repetition we measure is a mode of the policy rather than an im-

plementation defect. Second, **mathlib exposure cannot be excluded**. DeepSeek-Prover-V1.5-Base, which is in the ancestry above, was pre-trained on formal-language corpora including Lean [34]; Qwen3's pre-training corpus is not disclosed; and our base is supervised on text DeepSeek-Prover-V2 generated. Appendix B.9 carries this as a leakage channel, because it is the one no checking environment closes.

D.2 Outline of the thesis

Outline. Chapter 2 develops the two motivating problems in detail. Chapter 3 surveys related work on inference-time control of language models, learned guidance for proof search, failure-aware proving, and the reliability of formal-proof evaluation. Chapter 4 states the research questions and hypotheses. Chapter 5 presents the robust verification protocol, the probe architectures and where they read, the steering rules, and the training and selection procedure. Chapter 6 takes the three contributions in order: the reproduction audit, then probe generalization and what the signal is, then the pass@32 steering results and why this intervention does not help. Chapter 7 discusses limitations and future directions and concludes.

D.3 The approach in brief

Concretely, we train two probe architectures, a stateless multi-layer perceptron (MLP) head and a stateful long short-term memory (LSTM) head, each with under 3M parameters, on hidden states captured at proof-line starts during generation. We then use the probe as a *controller*: at each line start its failure probability p_{fail} edits the next-token distribution, either by rais-

ing the sampling temperature to $e^{\alpha p_{\text{fail}}}$ or by adding a probe-scaled frequency penalty, nudging the frozen prover away from the behaviors that precede failure. A validation contest chooses the probe and intervention for each base model. Crucially, the base prover is never retrained; the only learned component is the sub-3M-parameter probe, and the intervention adds negligible cost at inference.

D.4 Extended motivating discussion

Why simple heuristics are not enough. It is tempting to flag repetition with a hand-written rule, for example "abort after the same line appears n times". This fails because the prover's core strategy is to split a difficult goal into many similar sub-goals and discharge each with a similar tactic [23, 29]. That strategy is trained in rather than emergent: the model ours is distilled from was cold-started on skeletons of `have` sub-goals and rewarded for keeping every one of them in the final proof (§D.1). Repetition of a tactic is therefore ordinary in *successful* proofs too, and a surface rule cannot separate productive repetition from a stalled loop without understanding the proof state. This pushes the question one level deeper, from the emitted text to the model's internal representation: does the hidden state that the model computes while generating already encode whether the attempt is on track? Chapter 5 answers this by training a small probe on those states, and Chapter 6 shows the answer is largely yes.

This unreliability is not specific to one system. It recurs across the field: independent studies document REPL-accepted but incorrect proofs, contaminated "`sorry`"-bearing proofs treated as complete, and other scoring faults [2, 15, 18]. LeanTree, for example, finds 12 such REPL-accepted but

incorrect proofs in MiniF2F, a standard benchmark of competition problems [15]. The lesson we draw is methodological: no claim about proof success in this thesis, including the training labels for our probes, may rest on a persistent-REPL verdict. Every proof is recompiled from scratch and checked against the kernel's axioms. This robust verification protocol, defined in Chapter 5, is the measurement contract on which the rest of the thesis stands.

D.5 Extended related-work detail

Zhang et al. [38] train a binary classifier on hidden states at intermediate-answer positions and find it predicts correctness before the answer is fully written; used as a verifier for early exit, it cuts inference tokens by about a quarter without hurting accuracy. Davidov et al. [10] treat the same idea as a value estimate: an MLP probe on the hidden state at position t estimates the eventual value of the prefix, trained from terminal outcomes under a binary cross-entropy objective, and generation halts when that estimate falls below an abstention reward. Liang et al. [21] score the last-layer hidden state at reasoning-step boundaries, propagate each trace's final correctness label to all of its steps, fit a two-layer classifier under a class-weighted binary cross-entropy, and prune unpromising traces during generation.

The strongest current provers use much larger parameter, search, and interaction budgets than we do. Reinforcement-trained whole-proof and lemma-style provers, including Goedel-Prover-V2 [23], DeepSeek-Prover-V2 [29], Kimina-Prover [25], and Seed-Prover [7, 8], scale test-time sampling, refinement, and conjecture pools. A separate line wraps general-purpose closed models (Claude, GPT, Gemini) in tool-using Lean agents that iterate

against the compiler many times, such as Numina-Lean-Agent [24] and Ax-Prover [6]. Their strength depends heavily on the scaffold and its budget: without such a harness, the same closed models are far weaker at kernel-checked Lean. Work on adaptive test-time compute, such as EconProver, studies how to spend that budget more efficiently rather than uniformly [20].

Appendix E

Full Methods and Implementation

E.1 The Lean file a persistent REPL accepts

The following self-contained Lean file "proves" `False`. The definition `n` has kernel value 2 but compiled value 1 (via `@[implemented_by]`), so `decide` proves `n = 2` while `native_decide` proves `n = 1`, and `omega` closes `False`:

```
import Mathlib
import Aesop
set_option maxHeartbeats 0
open BigOperators Real Nat Topology Rat


def m : Nat := 1
@[implemented_by m] def n : Nat := 2
theorem bad : False := by
  have h1 : n = 2 := by decide
  have h2 : n = 1 := by native_decide
  omega
#print axioms bad
```

E.2 Pipeline and probe-head figures

These three figures accompany the method as it is described in §5.2 and §5.3; they are collected here so that the body can state the design without breaking for a full-width diagram. Figure E.1 draws the pipeline end to end: which states and labels are collected, what is trained on them, how a single steering arm is selected, and where verification sits.

The next two figures draw the probe heads themselves, layer by layer and in the same vocabulary, so that the one structural difference between them is visible at a glance. Figure E.2 is the stateless MLP head, whose input is the previous and current line-start hidden states concatenated.

Figure E.3 is the stateful LSTM head, whose input is one line-start hidden state at a time and which carries its own hidden and cell state across the line starts of an attempt.

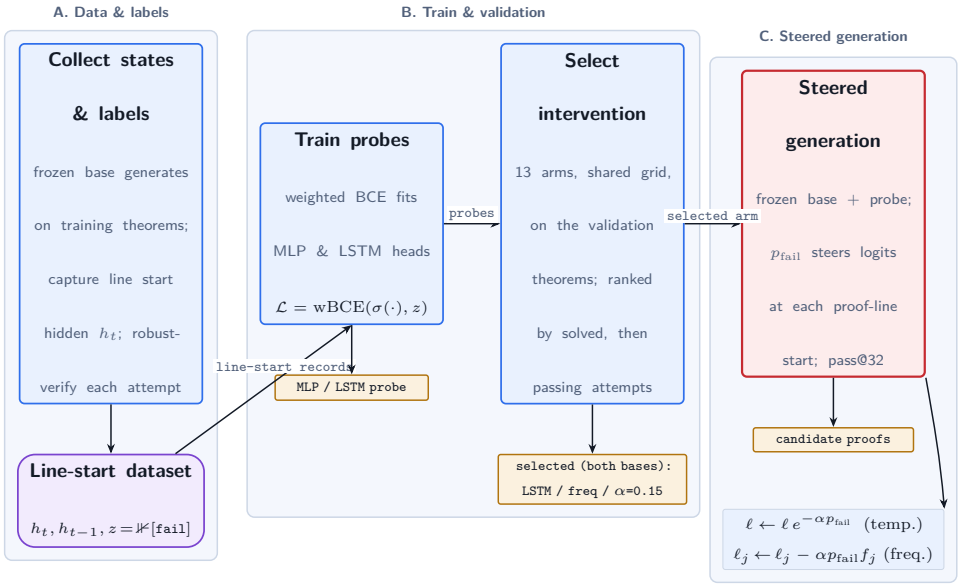


Figure E.1: **Probe-guided generation pipeline.** Only the probe (< 3M parameters) is trained; the prover weights are frozen. Symbols: h_t is the line-start hidden state, z the robust fail label, p_{fail} the probe's failure probability, ℓ the logits, α the steering strength, f_j the token frequency.

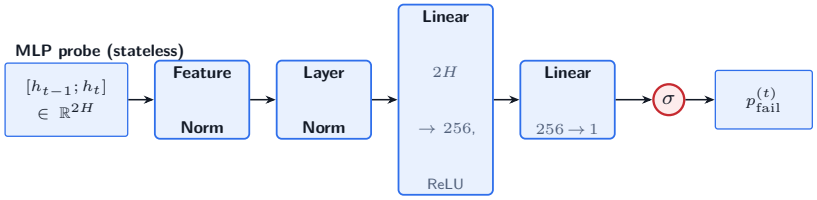


Figure E.2: **MLP probe head (stateless).** Its input is the previous and current proof-line-start hidden states concatenated, $[h_{t-1}; h_t] \in \mathbb{R}^{2H}$, with H the LLM hidden size (4096/5120 for 8B/32B). Feature normalization applies the mean and standard deviation stored in the checkpoint; LN is layer normalization, ReLU the rectified-linear activation and σ the logistic function. The head has 2.11/2.64M parameters for 8B/32B and emits the failure probability $p^{(t)}_{\text{fail}}$ for that line start.

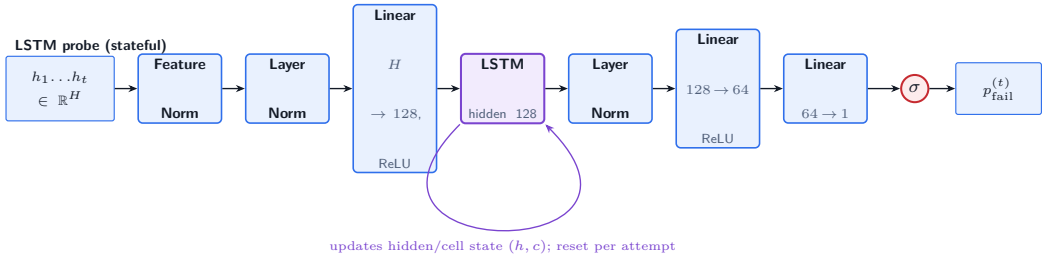


Figure E.3: **LSTM probe head (stateful).** Its input is the line-start hidden state h_t alone, $\in \mathbb{R}^H$, one line start at a time; the LSTM's own hidden and cell state (h, c) is updated at each line start and reset at the start of each attempt, so the head effectively sees the prefix $h_1 \ldots h_t$. H is the LLM hidden size (4096/5120 for 8B/32B); feature normalization applies the mean and standard deviation stored in the checkpoint, LN is layer normalization, ReLU the rectified-linear activation and σ the logistic function. The head has 0.67/0.81M parameters and emits the failure probability $p^{(t)}_{\text{fail}}$ for that line start.

E.3 Trainable parameters

Table E.1 gives the parameter counts quoted in §5.5. The base prover is used exactly as released and is never updated; the only weights any run of ours fits are those of a single probe head, and that head reads states the base has already computed.

Table E.1: Trainable parameters. Goedel-Prover-V2 is used as released; each run trains one small probe head on states already computed by the base.

Component (training?)	Goedel-Prover-V2-8B	Goedel-Prover-V2-32B
Base model (*frozen*)	8,190,735,360 (d=4096)	32,762,123,264 (d=5120)
MLP probe classifier (*trained*)	2,114,049	2,642,433
LSTM probe classifier (*trained*)	673,281	806,401

E.4 Tap-point statistics

The tap point was chosen *a priori*: it was fixed from the base model's own generations before any steered run existed, so no steering result could have informed it, and we report the property on those same base generations. Over every attempt of both base models, on the training theorems and on the test theorems, the token immediately following an indentation token inside the Lean block is real content, a tactic name or an identifier rather than further whitespace, in 89–97% of firings. The tap therefore lands where the reasoning above says it should, on both models and in both domains. That share is conditional on the reading already being inside the Lean block. Across the whole model output, including the planning prose that precedes the proof,

59% of readings on 8B and 70% on 32B sit immediately before real Lean content.

E.5 The epoch-selection score, and why it is not a result

1. **The epoch-selection score** is the one in the loop above, computed after every epoch to decide which epoch to keep. It reaches 0.98–0.99, and that is not a result. Its holdout is drawn at random from the training data rather than grouped by theorem, so line starts of the same proof, and for the MLP of the same attempt, sit on both sides of it; a probe can do well there by recognizing a proof it has already seen most of. It is also the largest of 30 evaluations on the very split that does the selecting. We give the number here only so that a reader who opens a checkpoint and finds it is not misled: it is a training control, and it is not comparable with either number below.

E.6 The training loop

E.6.1 The training loop

Both heads are fitted with Adam at a learning rate of 10^{-3} for 30 fixed epochs from seed 0, which takes about five minutes per head on a single GPU. The unit of a batch differs between them because the unit of an example does. For the MLP an example is one line start, and a batch is 512 of them; the training set is 115,285 readings on 8B and 177,552 on 32B. For the LSTM

an example is a whole attempt read in order, and a batch is 64 attempts padded to a common length with a mask, so the loss counts only real line starts and never the padding; that set is 1,805 attempts on 8B and 1,891 on 32B. The weight in the weighted loss is the ratio of verifying to failing examples among the training examples actually used, applied to the failing class: failures dominate, so the weight is below one and pulls them down, which is the same thing as pulling the rare verifying examples up. The per-feature mean and standard deviation are computed on the training line starts alone, saved inside the checkpoint and reused unchanged at inference, so the deployed controller sees exactly the inputs the head was fitted on. We use no early stopping, no learning-rate schedule, no weight decay and no gradient clipping. The labeled collection carries no separate validation split of its own, so after each epoch the head is scored on a 10% holdout drawn at random from the training data itself, readings for the MLP and whole attempts for the LSTM, and the weights we keep are the *best* epoch's rather than the last epoch's.

E.7 Why this α grid

Why this α grid. A whole-proof attempt is a long chain in which one tactic choice commits everything that follows, so the hypothesis under test is that a *light* nudge, applied at a few decision points, is enough to divert a doomed attempt onto a different branch. Changing which token is sampled at a handful of line starts changes the proof; rewriting the distribution is not required, and would test a different claim. We therefore chose the grid $\alpha \in \{0.15, 0.3, 0.5\}$ to stay inside that regime, and verified afterwards that it does. With the probe maximally confident (p_{fail}=1), the unsteered top token

carries 0.91 to 0.94 of the next-token mass; at α=0.15 the temperature rule removes about one point of that, and even at α=0.5, the strongest setting we search, the model's preferred token still holds 0.87 to 0.89. In effective-temperature terms the grid spans a 16% to 65% increase (Table E.2).

We deliberately did not search stronger settings. An attempt that stops being syntactically valid Lean fails for a reason that has nothing to do with the hypothesis, and would confound "steering changed the proof" with "steering broke the output". At the other end, below $\alpha \approx 0.05$ the frequency rule has no measurable effect: that is the smallest strength at which the steered distribution departs measurably from the unsteered one, and the temperature rule at that strength moves the top-token mass by well under a point, so smaller settings would not be measurable at K=32. The two rules are also not equally strong at equal α: at α=0.15 the frequency rule removes between 4 and 18 points of top-token mass where the temperature rule removes about one, which is worth keeping in mind when reading the contest outcome, since the contest selected the frequency rule.

This reasoning is *ex ante*, and the run bears it out. Three checks against the contest (Table E.3), all single-run. The unsteered base is not the best arm on either model, fifth of thirteen on 8B and twelfth of thirteen on 32B, so the grid is not a detour around α=0. The middle setting α=0.3 is the best of the three in four of the eight head-by-rule-by-base families and ties for best in a fifth, and only one family, the weakest on its base, improves monotonically toward α=0.5, so nothing in the sweep points above the grid. And at the selected setting the intervention measurably changes generation, cutting output by 21.5% and 14.0% (§6.8), so it is not a setting too small to act. The selected arm does sit at α=0.15, the lowest value searched, and the measurability bound above is the reason we did not search lower. Within-

family spreads reach at most seven theorems of ninety, so these checks bound the regime rather than identify an optimum. Separately (§6.8) the divergence from the succeeding distribution rises with α, which is a statement about that divergence and not about which setting proves more theorems.

E.8 Steering grid and validation contest

Two tables support the steering and selection sections of the body. Table E.2 restates the temperature intervention of §5.6 as an effective sampling temperature, which is the easier form in which to read off what a given α does.

Table E.2: **Effective temperature.** Multiplying the logits by $e^{-\alpha p_{\text{fail}}}$ is equivalent to raising the sampling temperature to $e^{\alpha p_{\text{fail}}}$; the last three rows show our α values at p_{fail}=1.

Logit multiplier	Effective temperature	Temperature increase
$e^{-\alpha p_{\text{fail}}}$	$e^{\alpha p_{\text{fail}}}$	$(e^{\alpha p_{\text{fail}}} - 1) \times 100\%$
$e^{-0.15}$	1.1618	16.18%
$e^{-0.3}$	1.3499	34.99%
$e^{-0.5}$	1.6487	64.87%

Table E.3 then gives every arm of the validation contest of §5.7, on both base models and against the unsteered reference.

Table E.3: **Validation contest results.** Theorems solved of 90 and passing attempts of 360 (K=4 per theorem), under the same target-blinded labeling used for training. *base* is the unsteered reference. Bold marks the selected arm, which is the same setting on both base models. The base is *not* the best arm here.

			8B		32B	
Probe	Rule	α	thms	atts	thms	atts
–	none (base)	0	40	101	38	98
MLP	temperature	0.15	39	92	39	104
MLP	temperature	0.3	41	98	42	111
MLP	temperature	0.5	35	81	39	103
MLP	frequency	0.15	38	91	41	104
MLP	frequency	0.3	41	95	42	113
MLP	frequency	0.5	41	97	41	107
LSTM	temperature	0.15	35	93	43	114
LSTM	temperature	0.3	36	89	44	115
LSTM	temperature	0.5	37	86	41	117
LSTM	**frequency**	**0.15**	**41**	**105**	**44**	**117**
LSTM	frequency	0.3	40	98	37	99
LSTM	frequency	0.5	39	90	43	99

E.9 Base architecture and the probe tap

Figure E.4 draws the base model as a layer stack and marks the point the probe reads from. The tap is the output of the final normalization, taken at each proof-line start; §5.8 describes it in the text, and the section that follows gives the implementation.

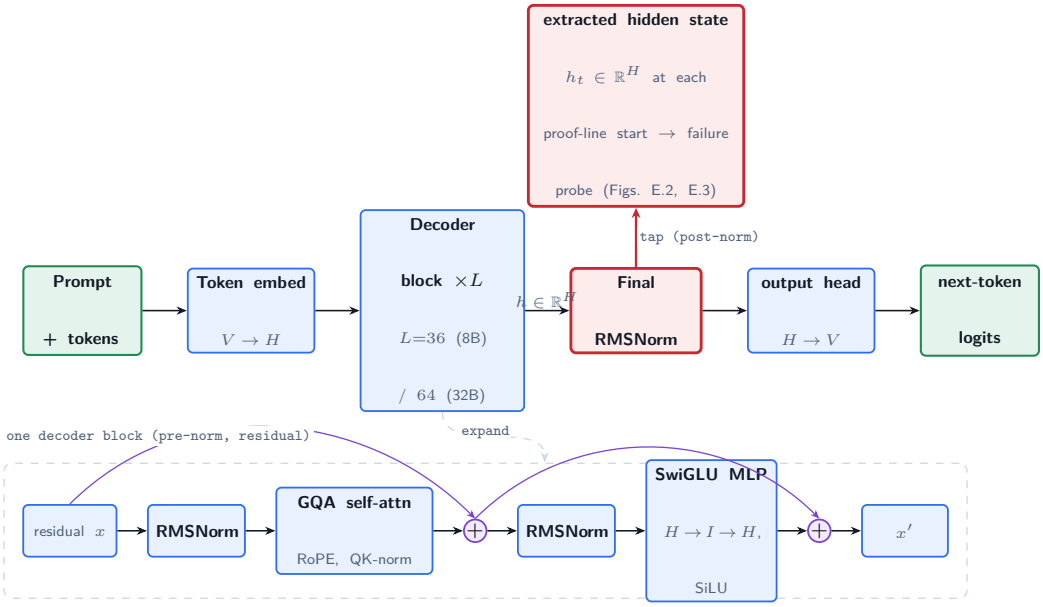


Figure E.4: **Base architecture and probe tap.** A Qwen3 dense decoder-only transformer (8B: L=36, H=4096; 32B: L=64, H=5120). The probe taps the final RMSNorm output (`model.model.norm`) at indentation tokens, just before the output head.

E.10 Base-architecture internals and extraction

Goedel-Prover-V2 fine-tunes the Qwen3 dense, decoder-only transformer; we use it *frozen* for generation and read its hidden states for features. The 8B base has L=36 layers, width H=4096, grouped-query attention (GQA) with 32/8 query/key–value heads; the 32B has L=64, H=5120 (64/8). Both use root-mean-square (RMS) normalization (ϵ=10^{-6}), rotary position embeddings (RoPE) with query/key normalization, a gated (SwiGLU) feed-forward block, and a separate output head. The probe consumes the *final* normalization output (`model.model.norm`), read at *indentation* tokens, the 229 vocab-

ulary items decoding to a whitespace run of two or more spaces: the model's internal state after the last block, just before the output head (Figure E.4). We use no logits, earlier-layer states, attention scores, or embeddings, and no extra forward pass is needed.

Extraction. During steered generation, a small hook inside the vLLM worker (on `compute_logits`) reads the post-normalization internal state and copies it to the CPU, one tensor per request, skipping the warm-up steps used for graph capture; the probe itself runs on the CPU. For one-time training-data collection we take the numerically identical vector from a HuggingFace forward pass (`model.model(input_ids).last_hidden_state`).

E.11 Probe training data

Table E.4 counts the training data described in §5.9: how many attempts each base model contributes, how many line-start readings those attempts yield, and what share of the readings comes from an attempt that verified.

Table E.4: **Probe training data** (filtered training split, target-blinded labels). A *reading* is one line start. Attempts with no line start contribute none, which is why the 8B attempt count is below its 1,850 labeled attempts.

	8B	32B
Theorems, filtered training split	243	243
with at least one verifying attempt	88	92
Attempts contributing readings	1,805	1,891
verifying	364	417
Proof-line readings, total	115,285	177,552
from failing attempts	106,691	165,212
from verifying attempts	8,594	12,340
share from verifying	7.5%	7.0%

E.12 The prompt, exactly as sent

On MathOlympiadBench the prompt has *five* parts: an instruction to complete a Lean 4 file; the Lean header, meaning the standard header plus any namespaces or imports that theorem needs; an informal statement of the problem as a comment; the formal theorem definition, ending in `:= by sorry`; and a closing instruction to give a proof plan before the formal proof. Every MathOlympiadBench theorem carries an informal statement, 359 of the 360 as a `/-!` module doc comment and one as a plain `/-` block. On the mathlib-derived training and validation theorems there are *four*: the same instruction, header and closing instruction, but a library lemma carries no informal statement, and its statement ends at its type rather than at `:= by sorry`. We checked both populations: no stored training prompt contains a comment block of any kind, and none ends in `:= by`.

````
Complete the following Lean 4 code:            <- 1. instruction

```lean4
import Mathlib                                 <- 2. Lean header
import Aesop
set_option maxHeartbeats 0
open BigOperators Real Nat Topology Rat

/-!                                            <- 3. informal statement
# IMO 2022 N8

Given n in |N such that 2^n + 65 divides
5^n - 3^n, prove that n = 0.
-/
theorem imo_sl_2022_N8                         <- 4. theorem definition
    (h : 5 ^ n = 3 ^ n [MOD 2 ^ n + 65]) : n = 0 := by sorry
```

Before producing the Lean 4 code, provide a   <- 5. plan, then proof
detailed proof plan outlining the main proof
steps and strategies. The plan should highlight
key ideas, intermediate lemmas, and proof
structures that will guide the construction of
the final formal proof.
````

(`|N` is $\mathbb{N}$ and the congruence sign is written `=` here; the arrows are annotations, not part of the prompt.)

E.13 Normalization and run metadata

Normalization. When a probe is trained we record, over the training data, the mean and standard deviation of each input feature and store them with

the weights; at inference we standardize every feature identically (subtract the mean, divide by the standard deviation, floored at a small ϵ so a near-constant feature cannot blow up the ratio), so the probe always sees inputs on the scale it was fit on. The probes read the hidden state alone; an earlier version also fed a normalized step index, which we removed.

The four deployed heads were fitted in a single array job on 1 September 2026, one L40S GPU per head and about five minutes each, under the loop of §E.6.1 with seed 0; each checkpoint records its own trainer, epoch count and seed, so the run that produced it can be identified from the file alone.

E.14 Reproducibility and released artifacts

The base models are the released Goedel-Prover-V2 8B/32B checkpoints; the only trained component is the sub-3M-parameter probe head, and every reported success uses the robust protocol of §5.1 in a Lean 4.9.0-rc1 mathlib4 workspace. Our robust verifier, the deployed probe checkpoints, and the per-attempt robust results and solved proofs for every arm are released at the companion repository `https://github.com/amitaminov-huji/Internal-State-Probes-for-Kernel-Verified-Theorem-Proving`.

Appendix F

Supporting Empirical Analyses

F.1 Probe diagnostics

Two figures support the empirical discussion. Figure F.1 tracks the probes' mean predicted probability of success against the generated-token index on MathOlympiadBench, separately for attempts that go on to verify and attempts that do not.

Figure F.2 turns from the probes to the budget. It gives the distribution of generated tokens per attempt under each robust verdict, and it is the measurement behind the waste described in §2.1.

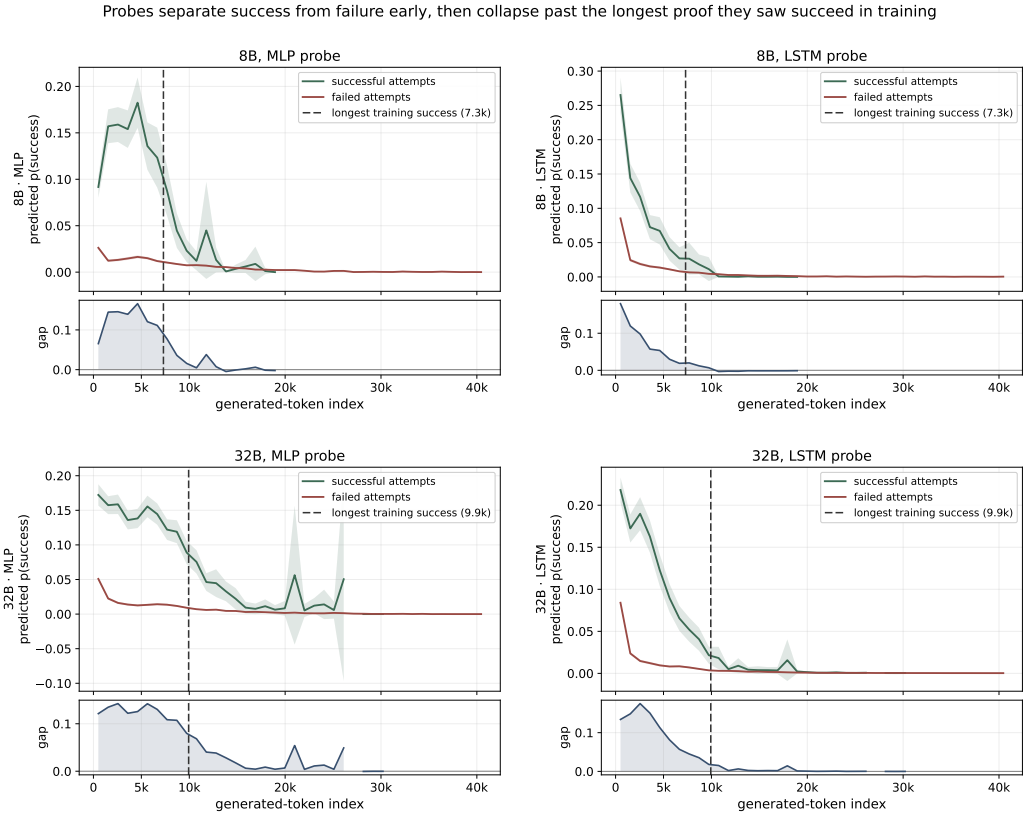


Figure F.1: **Probes separate success from failure early, then collapse past the longest proof they saw succeed in training.** Mean predicted p(success) against generated-token index on MathOlympiadBench, with 90% intervals; the strip below each panel is the success-minus-failure gap. Two readings must be kept apart. The curves sit *low* throughout, peaking at 0.09–0.27: out of domain the probes are badly *calibrated*, which is expected because the training theorems were easier and succeeded far more often. The *gap* between the curves is *discrimination*, and it is large early. The dashed line marks the longest successful training attempt (7,303 generated tokens on 8B, 9,932 on 32B); across it the mean gap falls by between 6× and 40×. It does not vanish there: the gap first reaches zero between 10,752 and 28,160 tokens, 1.5 to 2.9 times further out.

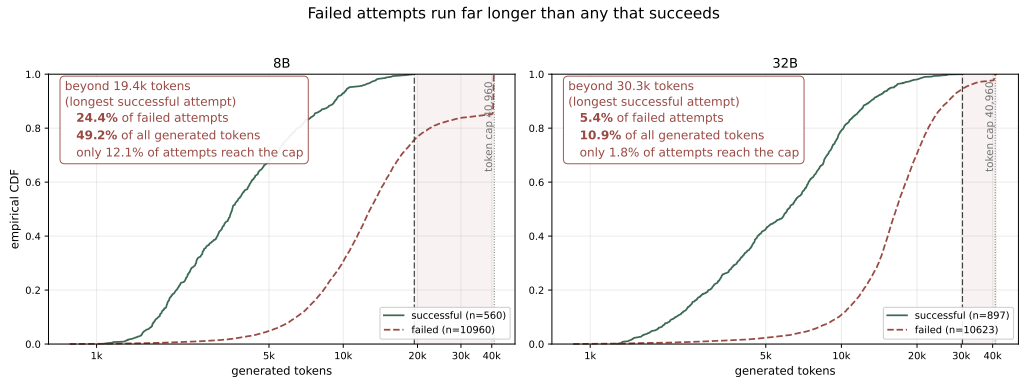


Figure F.2: **Failed attempts run far longer than any that succeeds.** Empirical CDF of generated tokens per attempt, MathOlympiadBench pass@32, base models, split by the robust verdict. The dashed line is the longest attempt that ever verified, 19.4k tokens on 8B and 30.3k on 32B; the shaded region is everything beyond it. Only 12.1% of 8B attempts and 1.8% of 32B attempts reach the 40,960-token cap, but 24.4% and 5.4% of failures run past anything that has ever succeeded, and those attempts consume 49.2% and 10.9% of all generated tokens. Compressibility is not plotted because it is largely a second view of the same length signal (§6.4).

F.2 Where generalization reaches, and where it stops

Figure F.1 shows where that generalization reaches and where it stops. The probes are uncalibrated out of domain, assigning low absolute success probabilities everywhere, while still separating the two populations clearly in the first few thousand tokens. The separation then collapses beyond the longest proof the probes ever saw succeed during training, which is the regime the training distribution never covered. Calibration and discrimination are dif-

ferent properties, and only the second is what a controller needs.

The probes read at indentation, which sits mostly inside the Lean block (66.6% of readings on 8B, 72.3% on 32B) and otherwise in the chain-of-thought, so the controller mainly acts on the proof while retaining some reach into the planning text that shapes it. Each reading is in any case built from everything the model has written so far, so it reflects the attempt's accumulated reasoning state and not only the local proof line, which may be why the signal transfers despite the difference in final-proof complexity. We do not isolate that mechanism.

F.3 Compressibility and the shape of the waste

It would be natural to read the DEFLATE ratio as measuring repetitiveness rather than size, but it does not: measured over the same attempts the DEFLATE ratio is anti-correlated with generated length at -0.89 on 8B and -0.69 on 32B, and the one roughly length-controlled statistic we compute, Huffman bits per byte, separates successes from failures by only 2%. Longer text compresses better because a dictionary has more to exploit, so the compression result is best read as a second view of the length signal rather than an independent one. Successful Lean proofs are short; failing ones ramble and loop. We report this as a finding in its own right rather than as an embarrassment: it says something true about the shape of machine-written proofs.

The cap is therefore the visible part of the waste rather than the large part. Most failing attempts stop well short of it and are still far longer than any

proof that has ever closed a goal: on 8B, nearly a quarter of failures run past the longest successful attempt and consume about half of the entire sampling budget (Figure F.2). That is the cost an early stopping signal would recover, and it is what motivates looking for one.

F.4 Off-the-shelf classifiers

Off-the-shelf classifiers are competitive. Fitting standard classifiers to a single hidden state, on exactly the same rows, a linear support vector machine reaches per-line ROC-AUC 0.87 on 8B against the probes' 0.83 and 0.83, so on the smaller base a simple model does better. On 32B nothing separates from our LSTM (0.87 versus 0.86, with gradient-boosted trees tied). The group is six classifiers, logistic regression, a linear support vector machine, an RBF support vector machine, a random forest, XGBoost and CatBoost. They are not strawmen: each is class-balanced against the 93:7 failure/success split and deliberately regularized. We did not run a hyperparameter search over them, so they are considered settings rather than tuned optima. The honest summary is that hidden states carry a real and usable failure signal, that our particular architectures are not the best way to read it on the smaller base, and that on the larger base they are competitive with everything we tried.

F.5 Validation and test, in detail

We do not read the in-domain margins as evidence that steering works. Neither is significant (paired exact McNemar p=1.00 on 8B and p=0.146 on 32B), both come from a single run, and K=4 on 90 theorems is a small com-

parison. What the reversal does show is the size of the domain gap, and it is the most direct evidence in this thesis for the claim that the probe is neither trained nor calibrated in the domain it is finally used in. The probe was fitted on mathlib proof lines and the arm was chosen on mathlib theorems; on olympiad problems the predicted probability of success peaks at 0.09–0.27 even on the attempts that go on to verify (§7.3), so the predicted failure probability is uniformly high and spans a narrow band, and an intervention scaled by it varies far less between attempts than their outcomes do. An in-domain gain that does not survive the move to the test domain is what a mis-calibrated controller looks like.

F.6 Which theorems change, case by case

The gains are, on 8B, `Imo1960P2` and `Imo1982P3b` (each from 0 to 2 verifying attempts of 32), and on 32B `Usa2001P3` (0→4), `Imo1979P5` (0→3), and `Usa1979P1`, `imo_sl_2007_A1a` and `imo_sl_2019_A1` (0→1 each). The losses are `Imo1964P2`, `Imo1968P5b`, `Imo1969P2`, `India1998P1b`, `Usa1980P5`, `Usa1993P1`, `Usa2001P3` and `imo_sl_2016_A5b` on 8B, and `Bulgaria1998P1`, `Imo1965P1`, `Imo1968P2`, `Imo1969P1`, `Imo2019P1` and `imo_sl_2010_A3` on 32B.

Two observations follow. First, the per-theorem picture is not a rescaled version of the aggregate: the same theorem, `Usa2001P3`, is a regression on 8B (1→0) and the largest gain on 32B (0→4), so the two base models do not respond to steering in the same way. Second, because each policy reaches theorems the other does not, a per-theorem *selector* between them has something to exploit, which we return to in §7.3. We stress that this is a statement about complementarity, not a result: running both policies costs twice the

budget, and whether the complementarity is stable across runs cannot be settled from a single draw.

F.7 Mechanism and divergence analyses

The intervention is strong enough. At the line starts the probe reads, the base model's next-token distribution is sharply peaked: entropy 0.27–0.38 nats (entropy measures how undecided the choice of the next token is; a *nat* is the natural-logarithm unit of that measure, with 0 meaning the next token is already fixed and 1 nat $\approx$ 1.44 bits. Values this small mean the model is effectively choosing among barely more than one token, since $e^{0.27}$ to $e^{0.38}$ is 1.3 to 1.5 options), about 0.9 of the mass on the single most likely token, an effective support of roughly 1.4 tokens, and a per-line elbow at rank 1 through the 75th percentile, so this is a property of the typical line start and not only of the average. Any *single* intervention at our strengths therefore almost always leaves the sampled token unchanged: at α=0.15 the temperature rule moves about one point of mass off the top token and the frequency rule between 4 and 18 points.

That does not make the intervention harmless, because it does not fire once. The probe fires at every indentation line start, on average 172 times per steered attempt on 8B and 255 on 32B, so over a whole attempt at least one changed token is close to certain, and a single changed token changes every line after it. The generations confirm it directly: steered runs are 21.5% and 14.0% shorter than the base. The α grid is therefore in the right regime, and light steering really is enough to divert attempts. **What fails is the direction of the change, not its magnitude**, which is what the next measurement establishes.

The effect is visible in the proofs that do succeed, not only in the aggregate token count (Figure F.3). Steered successful proofs are shorter and, at matched length, less repetitive. Both are exactly what a p_{fail}-scaled frequency penalty is built to do, and neither buys additional theorems.

The move is away from success-shaped generation. Measuring the Kullback–Leibler divergence between the failing attempts' next-token distribution after steering and the succeeding attempts' distribution, over exactly the grid the contest searched, with both sides taken as mean *rank profiles* (at each line start we sort the next-token distribution into decreasing probability order, its *decreasing rearrangement*, which is the usual convention for plotting a distribution against rank, and average those sorted vectors over line starts, so the comparison is between distribution *shapes* at matched ranks rather than between the probabilities of matched tokens; sorting leaves entropy unchanged, and we compare shapes because the tokens themselves differ from one line start to the next), the divergence *rises* with α on seven of the eight curves, and the eighth, the temperature rule on 8B out of domain, dips by under 0.001 nats at α=0.15 before rising steeply (Figure F.4). Flattening pushes failing generations away from the shape that succeeds rather than toward it. Read as a statement about this divergence alone, the minimizing setting in the searched range is $\alpha \to 0$ on those seven and α=0.15 on the eighth. That is not a statement about the contest ranking, where the unsteered base is *not* the best arm (Table E.3), and, for the reasons below, it is not a statement about which setting would prove more theorems. At the selected α=0.15 the frequency rule displaces the distribution between about 3 and 180 times further than the temperature rule does, depending on base and domain, and the frequency rule is the one the contest chose, which is

consistent with 8B, where it displaces most, losing most.

Pushing against reinforcement learning. The policy we are editing was distilled from DeepSeek-Prover-V2, including its 671B model, through scaffolded data synthesis and self-correction annotation, then supervised fine-tuned on Qwen3 and reinforcement-trained [23]. What that training put there can be named rather than gestured at (§D.1): a recursive `have` subgoal decomposition that its teacher was cold-started on, curriculum-trained on, and then rewarded by a consistency term for reproducing in full [29]. The same base is additionally averaged back toward its starting weights after every training stage, on the stated ground that training raises pass@1 while lowering pass@N and a prover that samples many attempts lives on that variety [23]; our intervention costs about a sixth of the verifying attempts at a fixed budget, which is an observation about that same axis rather than a mechanism we establish. A 0.67M-parameter probe fitted on 243 theorems can move that policy, but mostly away from what its training put there.

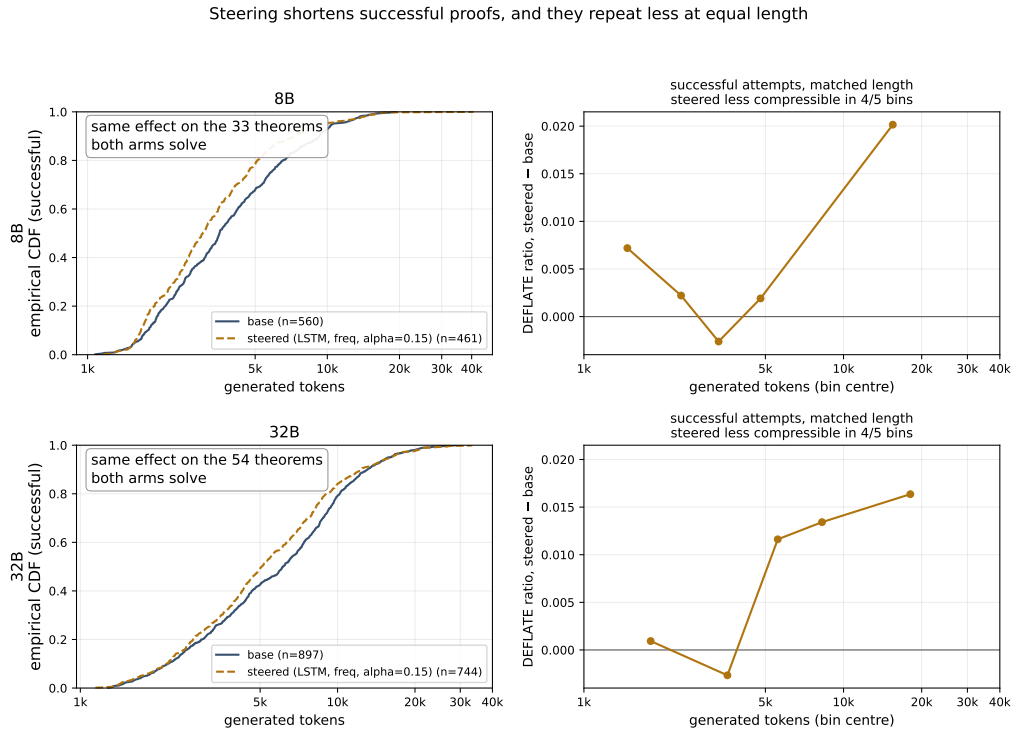


Figure F.3: **Steering shortens successful proofs, and they repeat less at equal length.** Successful attempts only. Left: the steered arm's median successful proof is 15–18% shorter (3,550 → 3,032 tokens on 8B, 6,159 → 5,080 on 32B); restricted to the 33 and 54 theorems *both* arms solve the effect is the same, so it is not an artifact of which theorems survived. Right: because shorter text is mechanically less compressible, the claim needs a control, so the DEFLATE ratio is compared inside matched length bins; steered output is less compressible in 4 of 5 bins on both bases, and the gap grows with length, which is what a frequency penalty should do. **This is evidence that the intervention acts, not that it helps**: the same arm closes fewer goals, 41 → 35 on 8B and 60 → 59 on 32B.

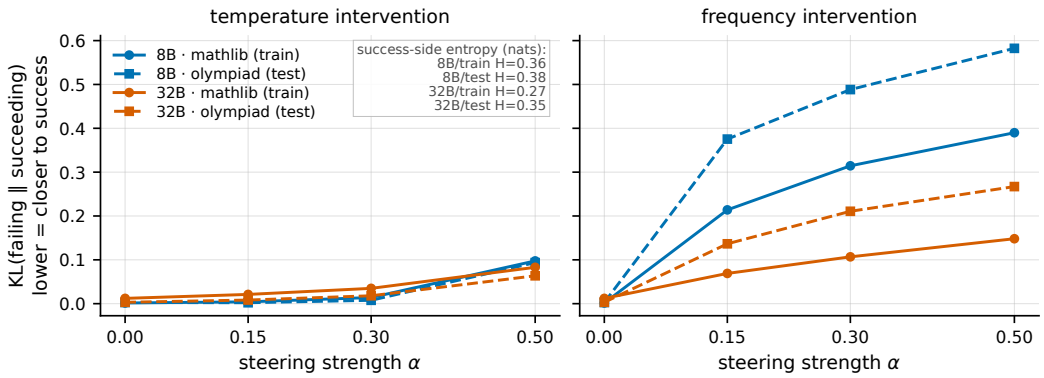


Figure F.4: **Steering moves failing generations away from success, not toward it.** Kullback–Leibler divergence between the failing attempts' next-token distribution after steering and the succeeding attempts' distribution, against steering strength α, over exactly the grid the validation contest searched. Lower would mean steering makes failing generations look more like succeeding ones; seven of the eight curves rise from α=0, and the eighth dips negligibly at α=0.15 before rising. Curves use an oracle probe (p_{fail}=1 on failed attempts); the trained MLP and LSTM curves lie essentially on top of them. Both sides are mean *rank profiles*: each next-token distribution is sorted into decreasing probability order before averaging, so the curves compare distribution shape at matched ranks rather than matched tokens.

Appendix G

Supplementary Discussion

G.1 Extended limitation details

These are the limitation items relocated from the body; the essential qualifications remain in §7.2.

- **One training theorem contributes nothing on 8B.** All eight 8B attempts at `Finset.card_product` prove the goal by `apply Finset.card_product`, that is, by applying the very lemma being proved, so the citation filter drops every one of them and the theorem contributes no labeled attempt and no readings. The 8B probe is therefore fitted on 242 of the 243 filtered training theorems; on 32B some attempts avoid the citation, so all 243 survive there. This is the citation filter doing its job rather than a defect, but it is an asymmetry between the two training sets and we report it.
- **A first-line-start mismatch found post hoc.** The MLP head's placeholder for "no previous line start" differs between training and inference,

a zero vector in raw hidden space versus in normalized space, and affects only the first line start of each attempt; the out-of-domain ROC-AUC is unchanged under both conventions at the precision we report, and the deployed head is the LSTM, which takes no explicit previous-state input.

- **The probe does not see every line.** A reading exists only where a line is indented by *three or more spaces*: the tokenizer merges the last space of an indent into the following word, so a two-space indent leaves only a single-space token, which the trigger excludes. Lines at column zero and lines at Lean's standard two-space tactic level therefore produce no reading and the controller is blind at those positions, 9.7% of proof lines in the verifying 8B attempts. Lean proof bodies are indented under `:= by`, so in practice this affects few lines, but it is a property of the trigger we chose rather than a deliberate design choice, and a differently-defined trigger would cover them.

- **Where the controller acts.** Indentation occurs wherever a line is indented, so the probe reads mostly inside the Lean block (66.6% on 8B, 72.3% on 32B, counted over every reading of all 11,520 base attempts per model) and otherwise in the chain-of-thought; a small part of that remainder, under 2% of all readings, sits in a Lean sketch the model wrote and then discarded rather than in prose. We record this as a property of the design rather than a defect: the controller principally acts on the proof it is trying to fix, while keeping some influence over the planning text that shapes it, and needs no region gate to do so.

- **A verification-header edge case, and a guard that never bound.** One validation theorem, `gcd_greatest`, is force-failed by our own checking header. The header emits the target's *complete* name; this theo-

rem is declared at root level, so that name carries no prefix, and under the header's `open Nat` it matches both the root-level `gcd_greatest` and `Nat.gcd_greatest`, so Lean errors on the attribute line before reading the proof. It is in neither the training set nor the test set and scored zero identically in all 13 contest arms, so it changes no reported result. It is also the *only* broken header, which we measured rather than argued: no training theorem shows an error on its attribute line in the stored compiles, and because a broken header fails every attempt of its theorem, we compiled the header of each validation theorem that scored zero in all 13 arms and found only this one. Our attribute-error guard did not flag it, because the guard matches `not registered`, `unknown identifier` and `unknown constant` but not `ambiguous identifier`. The guard has a second weakness we should state as plainly: it requires the target's name as a *quoted* identifier, and Lean writes identifiers as `'X'`, so a sibling ending in a prime makes the target's own quoted form a substring of an unrelated error. That is what its one firing in the whole project was, on `Ordinal.lift_lt` in a single 32B arm: a false positive on a proof-body error, with the attribute line compiling cleanly. The attempt was failing anyway, so **the guard changed no label and no count anywhere in this work**. We leave both weaknesses as they ran, since the labels are frozen.

- **Seven benchmark prompts the released builder damages.** Goedel-Prover-V2's released prompt builder cuts each prompt at the first `:= by`, which misfires on 7 of the 360 MathOlympiadBench theorems: five lose the statement to be proved, because an auxiliary `instance` or `def` carries a literal `:= by` before the theorem, and two end with a doubled terminator, because the theorem is written with a line break between `:=` and `by` so no

literal `:= by` is found. We keep the released prompt byte for byte, and the base and the steered runs were both decoded from it, so no comparison here is affected; six of the seven are never solved by any run.

G.2 Extended future-direction detail

Train and calibrate in domain. Our probes are fitted on 243 mathlib-derived theorems and deployed on olympiad problems. They discriminate well there, but they are not *calibrated* there: the predicted probability of success peaks at 0.09–0.27 even on the attempts that go on to verify, so the predicted failure probability is uniformly high and spans a narrow band, and an intervention scaled by it varies far less between attempts than their outcomes do. Figure F.1 also shows where the training distribution stops supporting them: across the longest proof either probe ever saw succeed in training, 7,303 generated tokens on 8B and 9,932 on 32B, the mean success-minus-failure gap falls by between 6× and 40×. The probes keep discriminating well past that point, so this is a collapse rather than an end, but the regime where the controller is weakest is precisely the regime the training corpus never covered. A larger, in-domain training corpus with calibrated outputs is the single change most likely to turn this result around, and it is the one we would try first.

A learned stopping policy. The most immediate use of a failure probe is not to bias generation but to *end* it: an attempt whose predicted failure probability stays high can be stopped and its budget spent on a fresh sample. This has two uses. Training-data generation currently spends much of its compute on attempts that run far past any proof that has ever closed a goal

(§2.1), and stopping them early would cut that cost: on 8B, 49.2% of all generated tokens go to attempts already longer than the longest successful one (Figure F.2). Test-time budget could shift away from attempts that have stalled rather than being fixed at k, in the spirit of adaptive-computation work [20]; outside formal proving, hidden-state estimates of eventual correctness already drive abstention and early exit [10, 38]. Our own experiments here used a simple threshold rule on probe outputs; a learned controller, together with an honest accounting against the alternative of simply sampling fewer attempts, is a research question in its own right.

Steer earlier. We intervene at proof-line starts, which is late: by then the model has committed to a proof line, and the next-token distribution there is close to deterministic. About a third of those line starts on 8B, and a quarter on 32B, already fall in the chain-of-thought, where the model is still planning and the distribution is less peaked. Steering during proof *planning* rather than during proof writing is the natural place to test the idea again, and our own measurements suggest it is a more promising target than the one we chose.

A contextualized router. Steering does not beat the base in aggregate, but the two policies are not nested: each proves a few theorems the other misses, and they do not even agree across base models, since `Usa2001P3` is a regression on 8B and the largest gain on 32B (§6.7). A small selector that estimates each policy's chance of success from the theorem, its domain and the proof context, and routes to the better one, could exploit that asymmetry. This resembles a mixture of experts, except that the two experts are one model run in two generation modes rather than separate networks, so it adds

almost no parameters. Whether the per-theorem complementarity is stable across runs, rather than an artifact of a single draw, must be established first, and our single-run data cannot settle it.